UNREAL ENGINE 5 • GAMEPLAY ABILITY SYSTEM

深入浅出 GAS
从原理、网络同步到实战

一本面向 UE 客户端 / 玩法程序的系统学习书：先建立正确的心智模型，再逐个拆解 Tag、Attribute、Effect、Ability、Task、Cue，然后把网络复制和客户端预测讲透，最后用一个完整的 MOBA 英雄把所有知识串起来。

基准版本 UE 5.4 – 5.6 | 15 章 + 4 篇附录 | 20+ 原理插图 | 2 个交互实验 | C++ 为主 · 兼顾蓝图

CHAPTER 00

阅读指南

这本书怎么读、读完能达到什么程度，以及书里各种标记的含义。

写给谁

本书假设你已经会写 UE C++：懂 `UCLASS` / `UPROPERTY`、Actor 生命周期，知道 Server/Client、RPC、属性复制大概是什么。**不要求你用过 GAS。**如果你已经用过 GAS、但一碰到"客户端扣蓝扣了两次""冷却 UI 抖动""眩晕打断不了技能""模拟端看不到特效"这类问题就没底，那么第三部分（网络同步）就是为你写的。

全书路线图

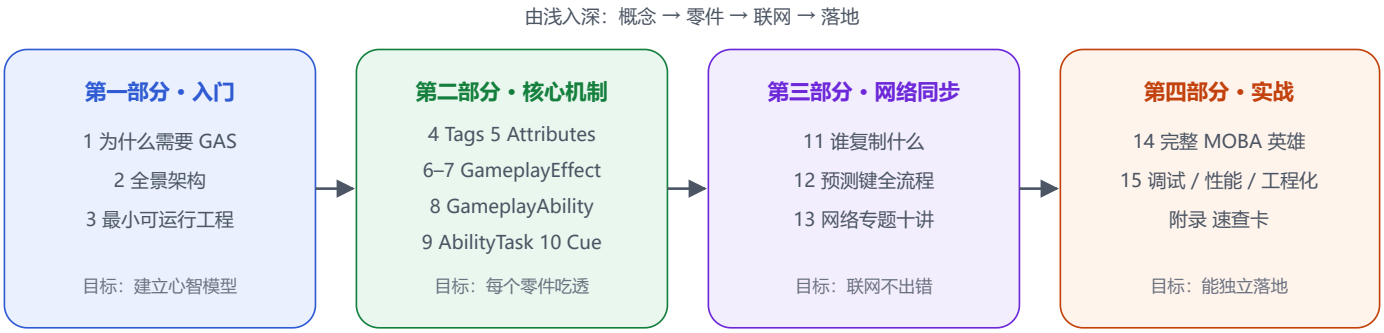


图 0-1 全书四个部分与推荐阅读路径

书中的标记

> ✅ **实践建议**
>
> 可以直接照做的工程经验。

⚠️ **坑点**

新手和老手都会踩的问题，通常对应一个真实 Bug。

🔬 **深入源码**

引擎内部是怎么做的。第一遍可以跳过，第二遍再看。

🌐 **网络视角**

这个知识点在联网时有什么不同。这是本书的重点，建议每处都读。

🕒 **版本差异**

UE4 / UE5 早期版本与 5.4+ 的 API 差异。

版本约定

正文以 **UE 5.4 – 5.6** 为准。和老教程（包括 tranek 的 GASDocumentation，它主要基于 UE4.27 / 5.0 时期）相比，有几处关键变化，读老资料时需要自己换算：

变化	起始版本	影响
GameplayEffect 改为**组件化**（GEComponents），Tag 相关字段迁移到各个 `U*GameplayEffectComponent`	5.3	老教程里 GE 面板上的 "GrantedTags / ApplicationTagRequirements" 等字段都改成了组件
`EGameplayModOp` 扩展为 AddBase / MultiplyAdditive / DivideAdditive / MultiplyCompound / AddFinal / Override	5.x 新版	聚合公式多了"复合乘"和"最终加"两项，见第 6 章
`NonInstanced` 实例化策略**废弃**，Ability 上的 Replicated 属性废弃	5.5	统一用 `InstancedPerActor`；Ability 内跨端传值改用 RPC
`ATTRIBUTE_ACCESSORS_BASIC` 宏进入引擎头文件	5.6	不必再从注释里复制 `ATTRIBUTE_ACCESSORS`

关于版本

GAS 是持续演进的插件。本书引用的类名、函数名都来自引擎源码与 Epic 官方文档。如果你所在的项目使用定制引擎，以项目里 `Engine/Plugins/Runtime/GameplayAbilities` 的实际源码为准，遇到对不上的地方，直接打开头文件看注释是最快的办法。

PART ONE

第一部分　入门：建立正确的心智模型

先回答三个问题：GAS 解决什么问题、由哪些零件组成、怎样在工程里跑起来。

CHAPTER 01

为什么需要 GAS

先看清楚问题，才能理解 GAS 那些乍看很"重"的设计是在解决什么。

本章目标

- 说清楚"不用 GAS 时，一个技能要写多少东西"
- 理解 GAS 的四条设计原则：数据驱动、组合优先、Tag 作为通用语言、网络内建
- 判断你的项目该不该用 GAS

1.1 一个火球术，要写多少代码？

策划的需求很简单："**按 Q 放火球，耗 30 蓝，冷却 6 秒，命中造成 100 点火焰伤害，并附加 3 秒燃烧，每秒 10 点。**"直接写的话，第一版大概是这样：

第一版：直接写在角色里

```
void AMyHero::CastFireball()
{
    if (Mana < 30.f) return;                              // 蓝量检查
    if (GetWorld()->GetTimeSeconds() < FireballReadyTime) return; // 冷却检查
    Mana -= 30.f;
    FireballReadyTime = GetWorld()->GetTimeSeconds() + 6.f;
    PlayAnimMontage(FireballMontage);
    SpawnFireball();                                      // 命中后 Target->TakeDamage(100)
}
```

看上去还行。但需求会一条条追加上来：

1. "**被眩晕时不能放技能。**"于是加 `bIsStunned`。之后又来了沉默、击飞、冰冻，每种控制对哪些技能生效还不一样。

2. **"装备 +20% 法强，Buff +30%，两者叠加。"**需要一套修饰器系统，还要能正确撤销：Buff 结束时减掉的值必须和当初加上的一样。
3. **"燃烧最多叠 3 层，重复命中刷新持续时间。"**要写 DOT 管理器，处理计时、层数和刷新规则。
4. **"冷却缩减 20%，冷却期间 UI 要显示转圈。"**冷却时长得能动态计算，UI 需要能查询剩余时间。
5. **"联网对战，延迟 100ms 时按键也要立即有反馈。"**这是最难的一条：客户端得先扣蓝、先播动画，服务器否决时还要回滚。

一个月后，`AMyHero` 里塞满了 bool、计时器和 RPC，每个技能都在重复写同样的检查。新人加一个"免疫控制"的 Buff，要改十几处 `if`。

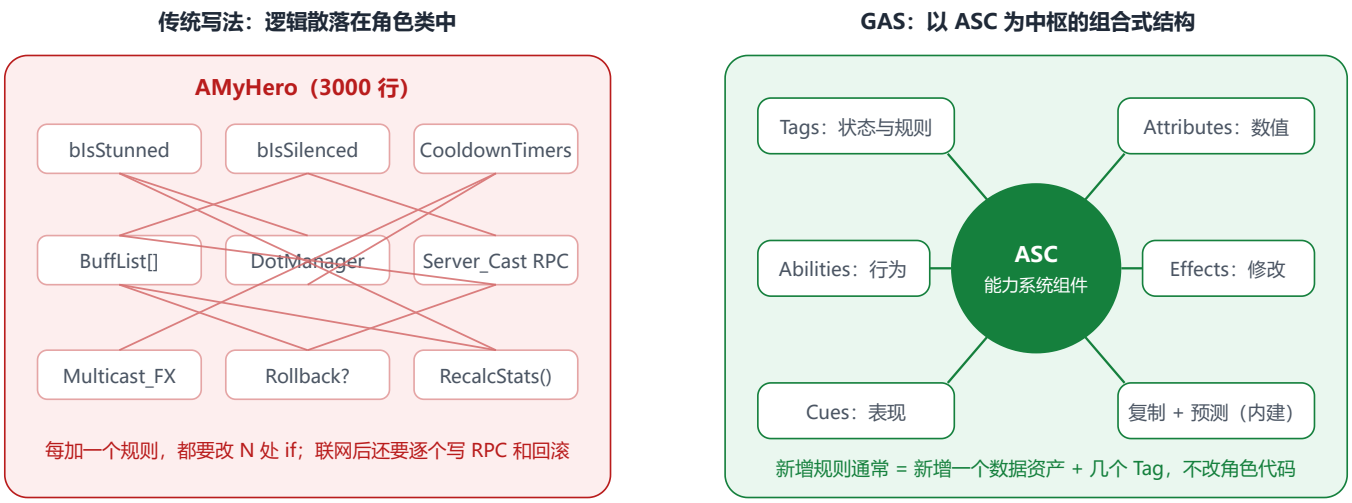


图 1-1 左：规则以 if/bool/RPC 的形式写在角色里，彼此耦合；右：GAS 把规则拆成五类可组合的数据对象

1.2 GAS 的四条设计原则

① 数据驱动

冷却、消耗、伤害、Buff 都写成**数据资产**（GameplayEffect），策划可以在编辑器里配置，不必改代码。

② 组合优于继承

"眩晕"不是一个类，而是"一个 GE 给目标加上 `State.Stunned` Tag"，加上"技能声明被这个 Tag 阻挡"。规则之间靠组合产生联系。

③ Tag 是通用语言

Ability、Effect、Cue、免疫、打断全部用层级化的 GameplayTag 描述，系统之间只依赖 Tag，不需要

④ 网络内建

属性复制、Effect 复制、技能激活 RPC、客户端预测与回滚都已经实现好。你只需要选择正确的策

互相引用类。

略。

用 GAS 重写火球术，**只需要一段几乎不变的 Ability 逻辑和几份数据**：

需求	GAS 中的表达方式	需要写代码吗
耗 30 蓝	Cost GE：Instant，Mana AddBase −30	**否**
冷却 6 秒（受冷却缩减影响）	Cooldown GE：持续 6 秒，授予 `Cooldown.Skill.Fireball`；时长用 SetByCaller 或 MMC 计算	**一次性**
眩晕 / 沉默时不能放	Ability 的 `ActivationBlockedTags` 填 `State.Debuff.Stun`、`State.Debuff.Silence`	**否**
+20% / +30% 法强叠加	装备与 Buff 各自是 Infinite / Duration GE 上的 Modifier，由聚合器自动计算，自动撤销	**否**
100 点火焰伤害（算抗性）	Damage GE + ExecutionCalculation（伤害公式写一次，全游戏复用）	**一次性**
燃烧 3 层、刷新时长	Burn GE：Duration 3s、Period 1s、Stacking 上限 3、刷新策略	**否**
命中特效、燃烧特效	GameplayCue：`GameplayCue.Fireball.Hit` / `GameplayCue.Burn`	**只做表现**
联网 + 按键立即反馈	Ability 设为 `LocalPredicted`	**否**

1.3 该不该用 GAS

适合

RPG、ARPG、MOBA、射击（Lyra、堡垒之夜都在用）、多人动作游戏。也就是技能、Buff 多，数值规则复杂，并且需要联网的项目。

慎用

只有一两个技能的单机小游戏；帧同步 / 锁步架构的项目（GAS 基于 UE 的 C/S 状态同步）；团队完全没有 C++ 能力（初始化必须写 C++）。

🌐 网络视角：与帧同步项目的关系

GAS 构建在 UE 的 **服务器权威 + 属性复制 + RPC** 模型之上，预测机制也是为这个模型设计的。帧同步项目（所有客户端用相同输入跑相同逻辑）一般不会直接采用 GAS 的网络部分，但它的**数据建模思想**（Tag、Modifier 聚合、Effect 生命周期、Stacking）可以直接借鉴到自研技能系统中。

本章小结

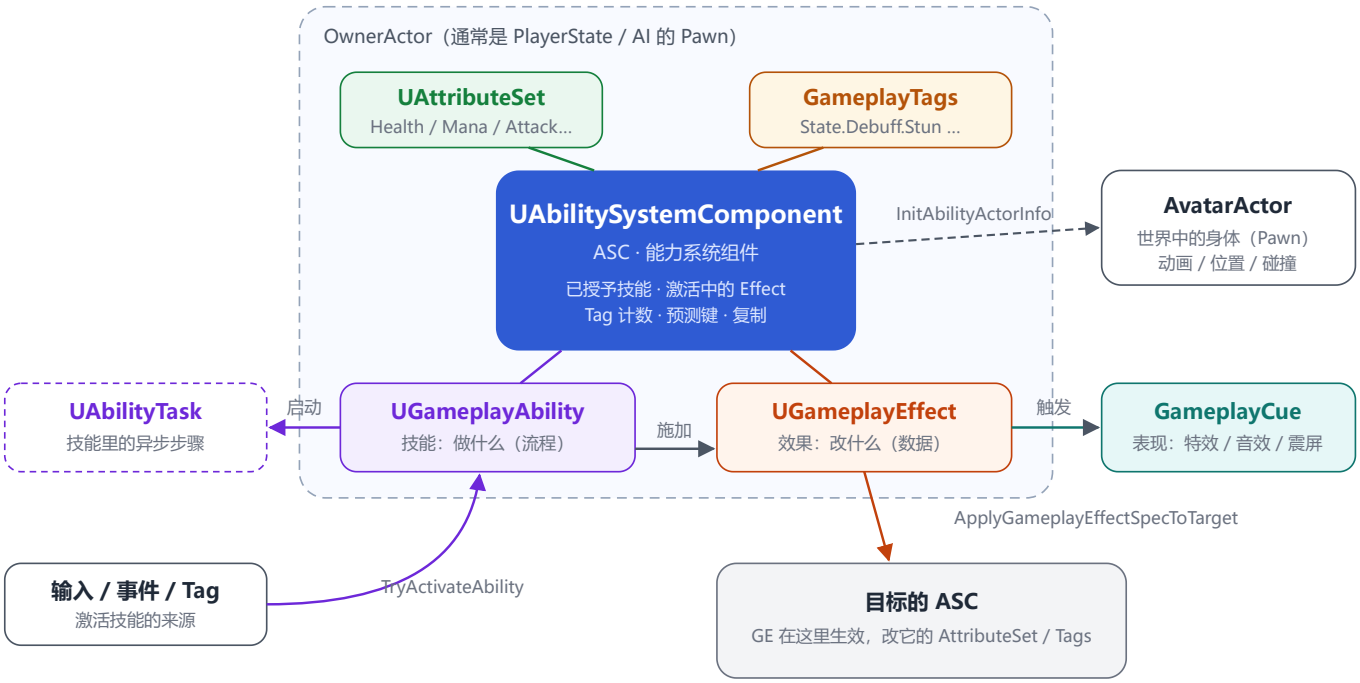
- 直接写技能时，难点不在单个技能，而在**规则之间的组合**和**联网后的同步与回滚。**
- GAS 把规则拆成 Tag / Attribute / Effect / Ability / Cue 五类可组合的数据，以 ASC 作为中枢。
- 大部分新需求只需要配数据；需要写代码的通常是一次性的框架部分（伤害公式、初始化、输入绑定）。

CHAPTER 02

全景架构：七个核心对象

先看整张地图，再逐个深入。这一章的图建议记住，后面每一章都是在放大其中一块。

2.1 一张图看懂 GAS



口诀：**Ability 决定"做什么"，Effect 决定"改什么"，Tag 决定"能不能"，Cue 决定"看起来怎样"**

图 2-1 GAS 全景：ASC 是中枢，挂在 OwnerActor 上，通过 AvatarActor 作用于世界

2.2 七个对象各是什么

对象	一句话	类比	典型例子
AbilitySystemComponent	挂在 Actor 上的组件，管理该 Actor 所有的 GAS 状态	角色的"战斗中枢"	每个英雄、小兵、防御塔各一个
AttributeSet	一组 float 属性的容器，带数值变化回调	角色面板	Health、MaxHealth、Mana、Armor
GameplayEffect (GE)	描述"怎么改属性 / 加什么 Tag / 持续多久"的**纯数据**资产	一张规范化的修改单	伤害、治疗、Buff、冷却、消耗
GameplayAbility (GA)	一段可激活的游戏逻辑，有完整的生命周期	行为脚本	普攻、冲刺、火球、开宝箱
AbilityTask	Ability 中的异步步骤（等动画、等事件、等输入）	协程中的 await	PlayMontageAndWait、WaitTargetData
GameplayTag	层级化的名字，例如 `A.B.C`	状态标签 / 系统间的通用语言	`State.Debuff.Stun`
GameplayCue (GC)	由 Tag 驱动的表现层回调，不影响逻辑	舞台灯光与音效	命中火花、燃烧粒子

2.3 定义、实例与句柄：贯穿全书的三层概念

GAS 的 API 第一眼很乱，主要原因是同一个概念有三种形态。弄清楚这一点，API 就好读了：



图 2-2 定义 → 实例 → 句柄。几乎所有 GAS API 都可以对应到这三层之一

> 🔬 **深入源码：Spec 存在哪里**
>
> 已授予的技能存放在 `ASC::ActivatableAbilities`（类型 `FGameplayAbilitySpecContainer`）中，正在生效的持续型效果存放在 `ASC::ActiveGameplayEffects`（类型 `FActiveGameplayEffectsContainer`）中。两者都是 `FFastArraySerializer`，也就是增量复制的数组，这是第 11 章的重点。

2.4 一次技能释放的完整旅程

把七个对象串起来，看一次"火球术命中敌人"在系统里依次经过哪些环节：

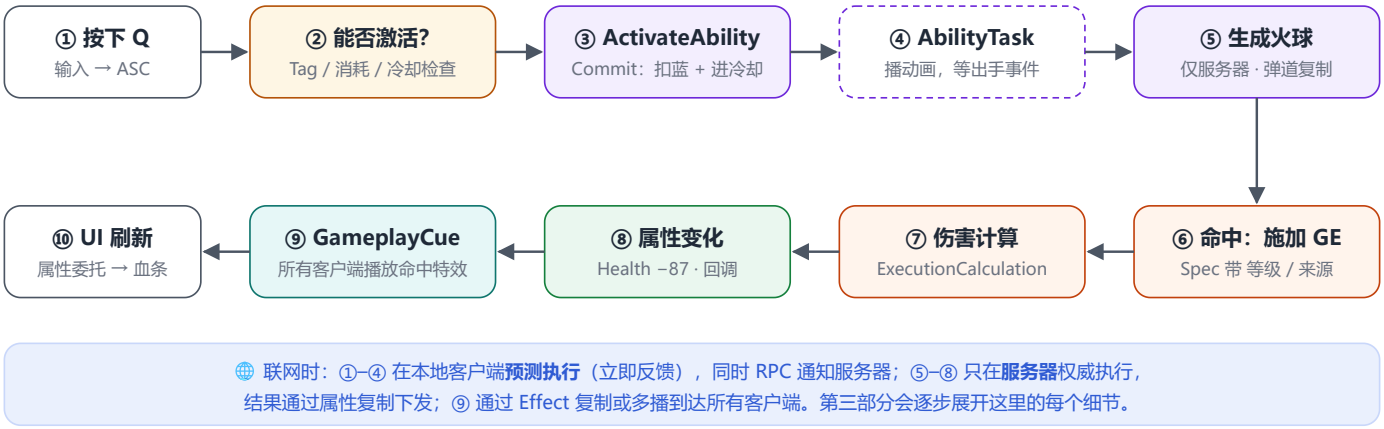


图 2-3 火球术从按键到血条变化的十个环节

2.5 逻辑与表现分离

GAS 有一条贯穿始终的界线：**GE、Attribute、Tag 是逻辑层**，由服务器权威决定；**GameplayCue 是表现层**，可以丢包、可以延迟、可以只在本地播放，丢了也不影响游戏结果。

> ⚠️ **坑点：不要在 Cue 里写逻辑**
>
> Cue 走的是 Unreliable 多播，而且只在客户端执行（专用服务器默认不跑表现）。在 Cue 里扣血、加 Buff，结果一定会不同步。反过来，Ability 也只存在于服务器和"拥有者客户端"，**在 Ability 里用 Multicast RPC 播特效是错误做法**，应该交给 Cue 去做。

本章小结

- ASC 是中枢；AttributeSet 存数值，GE 改数值，GA 写流程，Task 处理异步，Tag 管规则，Cue 管表现。
- 记住三层概念：**Class（定义）→ Spec（运行时实例）→ Handle（句柄）**。
- 逻辑在服务器权威执行，表现交给 Cue，二者不要混用。

CHAPTER 03

最小可运行工程

从零接入 GAS。这一章的每个决定（ASC 放哪、何时初始化）都直接影响后面的联网表现，不要跳过。

3.1 启用插件与模块依赖

在编辑器 *Edit → Plugins* 中启用 **Gameplay Abilities**，然后在模块的 `Build.cs` 中添加依赖：

MyGame.Build.cs

```
PublicDependencyModuleNames.AddRange(new string[] {
    "Core", "CoreUObject", "Engine", "InputCore", "EnhancedInput",
    "GameplayAbilities",    // ASC / GA / GE / AttributeSet
    "GameplayTags",         // FGameplayTag
    "GameplayTasks"         // AbilityTask 的基类 UGameplayTask
});
```

> 🕑 **版本差异：InitGlobalData**
>
> 老教程会要求在 `UAssetManager::StartInitialLoading()` 中手动调用 `UAbilitySystemGlobals::Get().InitGlobalData()`，否则 TargetData 等功能会报错。新版本的插件模块会自行完成全局初始化。如果你的项目是从 UE4 升级上来的，可以保留这行调用，它内部有防重复初始化的判断。

3.2 第一个关键决定：ASC 放在哪里

ASC 所在的 Actor 叫 **OwnerActor**，ASC 在世界中作用的"身体"叫 **AvatarActor**。两者可以是同一个 Actor，也可以分开：

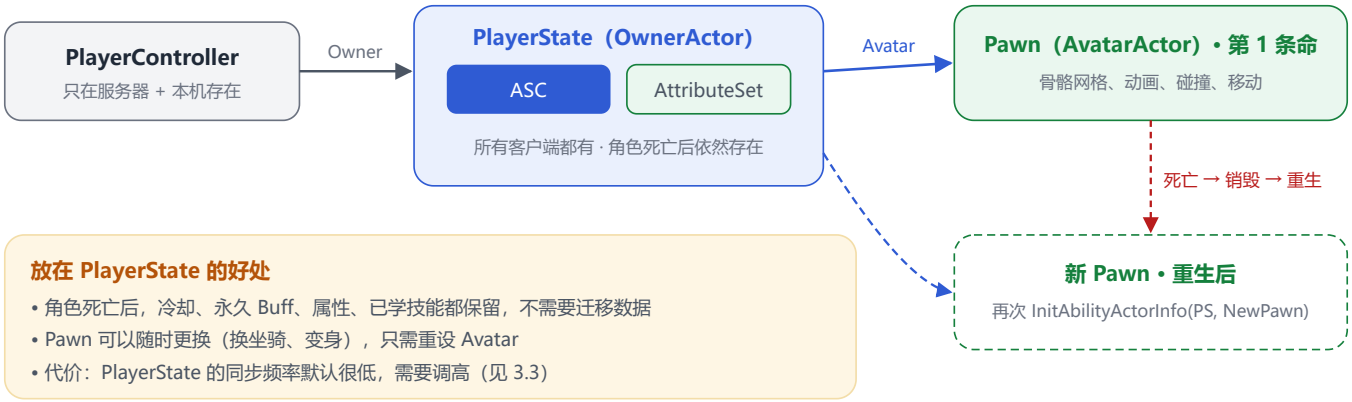


图 3-1 玩家角色推荐的结构：ASC 和 AttributeSet 放在 PlayerState，Pawn 只作为 Avatar

场景	ASC 放置	复制模式	说明
玩家角色（要保留状态）	PlayerState	Mixed	Lyra 与 Epic 官方最佳实践的推荐方案
玩家角色（每条命完全重置）	Pawn	Mixed	实现简单，死亡即清空
大量 AI 小兵	Pawn（Owner = Avatar = 自己）	Minimal	节省带宽，小兵不需要把 GE 细节同步给任何客户端
需要和玩家统一处理的 AI（Bot）	PlayerState（AIController 设 `bWantsPlayerState = true`）	Mixed	Bot 与玩家共用同一套代码
可破坏物、宝箱	自身	Minimal	同样可以被 GE 打
联网游戏	❌ **不要放 PlayerController**	—	PC 不会复制到其他客户端

> ⚠️ **坑点：一个 Actor 只放一个 ASC**
>
> ASC 初始化时会自动收集 OwnerActor 上所有 AttributeSet 子对象。放两个 ASC 会导致属性集被重复注册，`IAbilitySystemInterface` 的返回值也变得不确定。

3.3 PlayerState：创建 ASC 与 AttributeSet

MyPlayerState.h

```h
UCLASS()
class AMyPlayerState : public APlayerState, public IAbilitySystemInterface
{
    GENERATED_BODY()
public:
    AMyPlayerState();
    // IAbilitySystemInterface：让 GAS 以 O(1) 找到 ASC（未实现时会退化为遍历组件）
    virtual UAbilitySystemComponent* GetAbilitySystemComponent() const override { return Abili
    UMyAttributeSet* GetAttributeSet() const { return AttributeSet; }

protected:
    UPROPERTY(VisibleAnywhere, Category = "GAS")
    TObjectPtr<UMyAbilitySystemComponent> AbilitySystemComponent;

    // 必须用 UPROPERTY 持有，否则会被 GC
    UPROPERTY()
    TObjectPtr<UMyAttributeSet> AttributeSet;
};
```

MyPlayerState.cpp

```cpp
AMyPlayerState::AMyPlayerState()
{
    AbilitySystemComponent = CreateDefaultSubobject<UMyAbilitySystemComponent>(TEXT("AbilitySy
    AbilitySystemComponent->SetIsReplicated(true);
    // Mixed：GE 细节只同步给自己，其他客户端只收到 Tag 和 Cue（第 11 章详解）
    AbilitySystemComponent->SetReplicationMode(EGameplayEffectReplicationMode::Mixed);

    // 在构造函数中作为子对象创建：客户端无需等待复制就能拿到属性集
    AttributeSet = CreateDefaultSubobject<UMyAttributeSet>(TEXT("AttributeSet"));

    // PlayerState 默认同步频率很低，会导致技能/属性响应迟缓
    SetNetUpdateFrequency(100.f);   // UE5.5 之前写作 NetUpdateFrequency = 100.f;
}
```

> 🌐 **网络视角：为什么要调高 NetUpdateFrequency**
>
> 属性、ActiveGE 等通过**属性复制**下发，而属性复制受所在 Actor 的 `NetUpdateFrequency` 限制。PlayerState 默认的更新频率很低，不调高的话，服务器扣血后客户端要等很久才看到。ASC 的 RPC（激活、Cue）不受这个频率限制，所以表现为"特效出来了，血条半天才动"。

3.4 第二个关键决定：何时初始化 ActorInfo

`InitAbilityActorInfo(Owner, Avatar)` 会把 Owner、Avatar、PlayerController、动画实例、移动组件等缓存到 `FGameplayAbilityActorInfo` 中，技能运行时都从这里读取。**服务器和客户端必须各自调用一次**，而且要在双方都"准备好"之后调用：



图 3-2 两端初始化时机：服务器在 PossessedBy 中初始化，客户端在 OnRep_PlayerState 中初始化

MyCharacter.cpp —— 两端初始化

```cpp
// 服务器：Controller 附身时
void AMyCharacter::PossessedBy(AController* NewController)
{
    Super::PossessedBy(NewController);
    InitAbilitySystem();
    if (HasAuthority())
    {
        GrantStartupAbilitiesAndEffects();      // 技能只能在服务器授予
    }
}

// 客户端：PlayerState 复制到达时（此时 Pawn 也一定已经存在）
void AMyCharacter::OnRep_PlayerState()
{
    Super::OnRep_PlayerState();
    InitAbilitySystem();
}

void AMyCharacter::InitAbilitySystem()
{
    AMyPlayerState* PS = GetPlayerState<AMyPlayerState>();
    if (!PS) return;
    AbilitySystemComponent = PS->GetAbilitySystemComponent();    // 角色缓存一个指针，便于访问
    // Owner = PlayerState（持久），Avatar = 当前角色（身体）
    AbilitySystemComponent->InitAbilityActorInfo(PS, this);
}

// 角色同样实现 IAbilitySystemInterface，返回 PS 上的 ASC
UAbilitySystemComponent* AMyCharacter::GetAbilitySystemComponent() const { return AbilitySyste
```

MyPlayerController.cpp —— 兜底刷新 PC 缓存

```cpp
void AMyPlayerController::OnRep_PlayerState()
{
    Super::OnRep_PlayerState();
    // PC 可能晚于 PS/Pawn 到达；这里保证 ActorInfo 中缓存到了 PlayerController
    if (IsNetMode(NM_Client))
    {
        if (UAbilitySystemComponent* ASC = UAbilitySystemGlobals::GetAbilitySystemComponentFrc
        {
            ASC->RefreshAbilityActorInfo();
        }
    }
}
```

⚠️ **坑点：本地预测技能"按了没反应"**

症状是：Listen Server 上正常，纯客户端上 `LocalPredicted` 技能激活不了，或者全部走了服务器。原因通常是 `ActorInfo.PlayerController` 为空，ASC 认为自己"不是本地控制的"。按上面的写法补上 PC 端的 Refresh 即可。Lyra 在 5.6 中也修复了同样的问题。

🌐 **网络视角：OwnerActor 的 Owner 链**

客户端要能调用 ASC 上的 Server RPC，ASC 所在 Actor 的 **NetOwner 必须是该客户端的连接。** PlayerState 的 Owner 默认就是 PlayerController，天然满足；ASC 放在 Pawn 上时，Possess 会把 Pawn 的 Owner 设为 Controller，同样满足。**Mixed 复制模式还额外要求 OwnerActor 的 Owner 是 Controller**，否则 GE 无法正确同步给拥有者。

3.5 AI / 小兵：Owner 与 Avatar 是同一个 Actor

MyMinion.cpp

```
AMyMinion::AMyMinion()
{
    AbilitySystemComponent = CreateDefaultSubobject<UAbilitySystemComponent>(TEXT("ASC"));
    AbilitySystemComponent->SetIsReplicated(true);
    AbilitySystemComponent->SetReplicationMode(EGameplayEffectReplicationMode::Minimal); // 小
    AttributeSet = CreateDefaultSubobject<UMyAttributeSet>(TEXT("AttributeSet"));
}

void AMyMinion::BeginPlay()
{
    Super::BeginPlay();
    AbilitySystemComponent->InitAbilityActorInfo(this, this);   // 两端都在 BeginPlay 初始化即可
    if (HasAuthority()) { GrantStartupAbilitiesAndEffects(); }
}
```

3.6 授予技能与初始化属性

授予初始技能 / 初始属性（只在服务器）

```
void AMyCharacter::GrantStartupAbilitiesAndEffects()
{
    UAbilitySystemComponent* ASC = GetAbilitySystemComponent();
    if (!ASC || bStartupGranted) return;      // 重生时 ASC 在 PS 上，避免重复授予

    for (const TSubclassOf<UGameplayAbility>& AbilityClass : StartupAbilities)
    {
        // Spec(类, 等级, InputID, SourceObject)
        ASC->GiveAbility(FGameplayAbilitySpec(AbilityClass, 1, INDEX_NONE, this));
    }

    // 用一个 Instant GE（Override 各项属性）初始化属性，而不是直接 Set
    FGameplayEffectContextHandle Ctx = ASC->MakeEffectContext();
    Ctx.AddSourceObject(this);
    FGameplayEffectSpecHandle Spec = ASC->MakeOutgoingSpec(GE_InitAttributes, CharacterLevel,
    ASC->ApplyGameplayEffectSpecToSelf(*Spec.Data.Get());

    bStartupGranted = true;
}
```

✅ **实践建议：属性初始化的三种方式**

- **Instant GE + Override**：最直观，可以按等级读曲线表，推荐入门使用。
- **DataTable**：`InitStats(AttributeSetClass, DataTable)`，简单但不支持等级。
- **FAttributeSetInitter + CurveTable**：堡垒之夜使用的方式，在项目设置 `GlobalAttributeSetDefaultsTables` 中配置曲线表，按"组名 + 等级"初始化，适合有大量单位的项目。

3.7 输入绑定：Enhanced Input + InputTag

引擎内置的 `BindAbilityActivationToInputComponent` 依赖旧版输入系统和一个枚举，大项目一般不用。现在的主流做法（Lyra）是**让技能在 DynamicTags 中声明自己响应哪个 InputTag**，输入组件只负责把 InputTag 转发给 ASC：

MyAbilitySystemComponent.cpp —— InputTag 驱动

```
// 授予时：Spec.GetDynamicSpecSourceTags().AddTag(InputTag_Ability_Q);  (5.5 之前为 Spec.Dynami

void UMyAbilitySystemComponent::AbilityInputTagPressed(const FGameplayTag& InputTag)
{
    for (FGameplayAbilitySpec& Spec : GetActivatableAbilities())
    {
        if (!Spec.Ability || !Spec.GetDynamicSpecSourceTags().HasTagExact(InputTag)) continue;

        AbilitySpecInputPressed(Spec);           // 让 WaitInputPress 等 Task 能收到
        if (Spec.IsActive())
        {
            // 技能已经在运行：把"按下"事件同步给服务器上的同一个技能实例
            InvokeReplicatedEvent(EAbilityGenericReplicatedEvent::InputPressed,
                                  Spec.Handle, Spec.GetPrimaryInstance()→GetCurrentActivatior
        }
        else
        {
            TryActivateAbility(Spec.Handle);     // 会根据 NetExecutionPolicy 自动走预测/RPC
        }
    }
}
```

🕑 版本差异

`FGameplayAbilitySpec::DynamicAbilityTags` 在 5.5 中改为通过 `GetDynamicSpecSourceTags()` 访问；`GetPrimaryInstance()` 是 5.5 之后获取 InstancedPerActor 实例的推荐接口（之前一般用 `GetAbilityInstances()` 取最后一个）。旧接口仍可编译，但会给出废弃警告。

3.8 验证：showdebug abilitysystem

运行游戏，在控制台输入 `showdebug abilitysystem`，屏幕上会列出当前目标的属性、Owned Tags、已授予技能和激活中的 GE。用 `AbilitySystem.Debug.NextCategory` 切换显示类别，用 PageUp / PageDown 切换查看对象。看到属性值和技能列表，说明接入已经成功。

本章小结

- 玩家的 ASC 推荐放在 **PlayerState**（Mixed 模式，调高 NetUpdateFrequency），AI 小兵放在自身（Minimal 模式）。
- `InitAbilityActorInfo` 两端都要调用：服务器在 `PossessedBy`，客户端在 `OnRep_PlayerState`，PC 端再做一次 Refresh 兜底。
- 技能**只能在服务器授予**；属性用 GE 初始化；输入用 InputTag 转发给 ASC。

▶ **思考题：ASC 放在 PlayerState 上，角色死亡时"正在燃烧"的 DOT 该怎么处理？**

PART TWO

第二部分　核心机制：逐个拆解

从最基础的 Tag 和 Attribute 开始，逐层往上讲到 Effect、Ability、Task 和 Cue。每章都配有源码级别的细节和可直接使用的代码。

CHAPTER 04

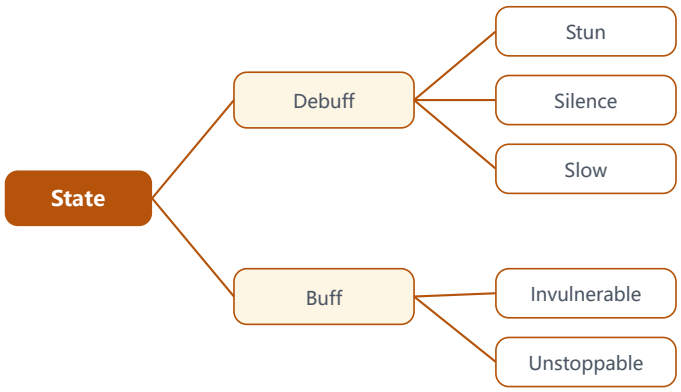
GameplayTags：系统之间的通用语言

Tag 是 GAS 中最简单、也最重要的概念。Tag 体系设计得好，后面大部分规则可以只靠配置完成。

4.1 Tag 是什么

`FGameplayTag` 本质上是一个注册过的 `FName`，用点号表示层级，例如 `State.Debuff.Stun`。它比 FName 和枚举更适合描述状态，原因有三点：

- **层级匹配**：查询 `State.Debuff` 时，`State.Debuff.Stun` 也会命中。"驱散所有负面效果"只需要一行。
- **全局注册**：所有 Tag 都在 TagManager 中注册，编辑器里可以下拉选择，拼错名字会在加载时报错。
- **网络友好**：复制时传的是索引而不是字符串（开启 Fast Replication 后），带宽开销很小。



角色身上的 Tag：State.Debuff.Stun

`HasTag(State.Debuff.Stun)`	✓
`HasTag(State.Debuff)`	✓ 父级命中
`HasTag(State)`	✓ 父级命中
`HasTagExact(State.Debuff)`	✗ 精确匹配
`HasTag(State.Debuff.Silence)`	✗

方向很重要：持有**子** Tag，查询**父** Tag 会命中；
持有父 Tag，查询子 Tag **不会**命中。

在 Project Settings → GameplayTags 或 C++ 中定义

图 4-1 Tag 的层级结构与匹配规则：HasTag 会向上匹配父级，HasTagExact 只做精确匹配

4.2 定义 Tag 的三种方式

方式	适合	备注
Project Settings → GameplayTags（写入 `Config/DefaultGameplayTags.ini`）	策划配置使用的 Tag	可以拆成多个 `Config/Tags/*.ini` 文件，减少多人修改同一文件时的冲突
DataTable（行结构 `GameplayTagTableRow`）	批量导入	在 GameplayTagTableList 中注册
Native Tag（C++）	代码中需要直接引用的 Tag	推荐。编译期可检查，不用担心字符串拼错

MyGameplayTags.h / .cpp —— Native Tag

```
// .h
#include "NativeGameplayTags.h"
namespace MyTags
{
    UE_DECLARE_GAMEPLAY_TAG_EXTERN(State_Debuff_Stun);
    UE_DECLARE_GAMEPLAY_TAG_EXTERN(Ability_Type_Attack);
    UE_DECLARE_GAMEPLAY_TAG_EXTERN(Event_Montage_Hit);
    UE_DECLARE_GAMEPLAY_TAG_EXTERN(SetByCaller_Damage);
}

// .cpp
namespace MyTags
{
    UE_DEFINE_GAMEPLAY_TAG_COMMENT(State_Debuff_Stun, "State.Debuff.Stun", "眩晕：无法移动与施法
    UE_DEFINE_GAMEPLAY_TAG(Ability_Type_Attack, "Ability.Type.Attack");
    UE_DEFINE_GAMEPLAY_TAG(Event_Montage_Hit,   "Event.Montage.Hit");
    UE_DEFINE_GAMEPLAY_TAG(SetByCaller_Damage,  "SetByCaller.Damage");
}

// 使用：
if (ASC->HasMatchingGameplayTag(MyTags::State_Debuff_Stun)) { /* ... */ }
```

4.3 容器与查询

API	含义
`FGameplayTagContainer::HasTag(T)`	容器中有 T 或 T 的子 Tag 就返回 true
`HasTagExact(T)`	只做精确匹配
`HasAny(Other)` / `HasAll(Other)`	与另一个容器做"任一 / 全部"匹配（层级匹配）
`FGameplayTagQuery`	可组合的布尔表达式，例如"包含 A 且不包含 B"，常用于 GE 查询
`FGameplayTagRequirements`	RequireTags + IgnoreTags（5.x 还有 TagQuery），GE 和 Modifier 的条件判断都使用它

4.4 ASC 上的 Tag：带计数

ASC 内部通过 `FGameplayTagCountContainer` 记录每个 Tag 的**引用计数**。两个 GE 都授予了 `State.Debuff.Slow`，计数为 2；移除其中一个后计数为 1，Tag 依然存在。这样不同来源的同一状态就不会互相覆盖。

Tag 来源	如何添加	如何移除	是否复制
GE 授予（GrantedTags）	GE 上加 `TargetTagsGameplayEffectComponent`	GE 结束时自动移除	**随 GE / Minimal Tags 复制**
技能激活期间	GA 的 `ActivationOwnedTags`	技能结束时自动移除	**各端在本地激活时各自添加**
Loose Tag	`AddLooseGameplayTag`	必须手动移除	**不复制**
复制的 Loose Tag	`AddReplicatedLooseGameplayTag`（较早版本为 `AddMinimalReplicationGameplayTag`）	对应的 Remove 函数	**复制**

监听 Tag 变化：眩晕时打断技能

```
ASC->RegisterGameplayTagEvent(MyTags::State_Debuff_Stun, EGameplayTagEventType::NewOrRemoved)
    .AddUObject(this, &AMyCharacter::OnStunTagChanged);

void AMyCharacter::OnStunTagChanged(const FGameplayTag Tag, int32 NewCount)
{
    if (NewCount > 0 && HasAuthority())
    {
        // 打断除"被动"以外的所有技能
        FGameplayTagContainer Ignore(MyTags::Ability_Type_Passive);
        GetAbilitySystemComponent()->CancelAbilities(nullptr, &Ignore);
    }
}
```

网络视角：Tag 计数只在服务器可靠

Mixed / Minimal 模式下，其他客户端（Simulated Proxy）上的 Tag 来自 `MinimalReplicationTags`，**只同步"有没有"，不同步"有几个"**。所以不要把 Tag 计数当作玩法计数器（比如层数），层数应该用 GE 的 StackCount 或属性表示。

4.5 推荐的 Tag 命名体系

根节点	用途	示例
`Ability.*`	技能身份 / 类型（技能的 AssetTags）	`Ability.Hero.Q`、`Ability.Type.Attack`、`Ability.Type.Passive`
`State.*`	角色状态（被检查的条件）	`State.Dead`、`State.Debuff.Stun`、`State.Casting`
`Cooldown.*`	冷却 GE 授予的 Tag	`Cooldown.Hero.Q`
`Effect.*`	GE 自身的分类（AssetTags），用于驱散、免疫、查询	`Effect.Type.CC`、`Effect.RemoveOnDeath`
`Event.*`	GameplayEvent	`Event.Montage.Hit`、`Event.Hit.Received`
`Damage.Type.*`	伤害类型	`Damage.Type.Physical`、`Damage.Type.Magic`
`SetByCaller.*`	运行时传入的数值键	`SetByCaller.Damage`、`SetByCaller.Cooldown`
`GameplayCue.*`	表现（**必须**以此开头）	`GameplayCue.Hero.Q.Hit`
`InputTag.*`	输入映射	`InputTag.Ability.Q`

✅ **实践建议**

Tag 改名时，在 `DefaultGameplayTags.ini` 中加 `+GameplayTagRedirects=(OldTagName="...",NewTagName="...")`，否则已经引用旧 Tag 的资产会失效。开启 `FastReplication` 后，客户端和服务器的 Tag 列表必须完全一致。

本章小结

- Tag 是层级化、全局注册的名字；`HasTag` 会向上匹配父级，`HasTagExact` 只做精确匹配。
- ASC 上的 Tag 带引用计数，可以来自 GE、技能激活或 Loose 添加；**Loose Tag 默认不复制。**

- 先规划好 Tag 树（Ability / State / Cooldown / Effect / Event / GameplayCue），后面大部分规则都能靠配置完成。

CHAPTER 05

Attribute 与 AttributeSet：数值的家

理解 Base 与 Current 的区别、回调的调用顺序，以及 Meta 属性的用法，是写好伤害和数值系统的前提。

5.1 BaseValue 与 CurrentValue

每个属性是一个 `FGameplayAttributeData`，里面**同时保存两个 float**：



图 5-1 Base 是永久值，Current = Base 经过所有持续型 Modifier 聚合后的结果

> ⚠️ **坑点：不要把"最大生命值"当作 Health 的 Base**
>
> 新手常以为 Health 的 Base 就是满血值。正确的建模是：`Health` 和 `MaxHealth` 是**两个独立属性**，`Health` 被 Instant 伤害或治疗修改 Base，`MaxHealth` 由装备、Buff 等持续型 GE 修改 Current。

5.2 编写 AttributeSet

MyAttributeSet.h

```h
#include "AttributeSet.h"
#include "AbilitySystemComponent.h"

// UE5.6 起引擎内置 ATTRIBUTE_ACCESSORS_BASIC；更早的版本自行定义下面这个宏
#define ATTRIBUTE_ACCESSORS(ClassName, PropertyName) \
    GAMEPLAYATTRIBUTE_PROPERTY_GETTER(ClassName, PropertyName) \
    GAMEPLAYATTRIBUTE_VALUE_GETTER(PropertyName) \
    GAMEPLAYATTRIBUTE_VALUE_SETTER(PropertyName) \
    GAMEPLAYATTRIBUTE_VALUE_INITTER(PropertyName)

UCLASS()
class UMyAttributeSet : public UAttributeSet
{
    GENERATED_BODY()
public:
    UMyAttributeSet();

    UPROPERTY(BlueprintReadOnly, Category = "Vital", ReplicatedUsing = OnRep_Health)
    FGameplayAttributeData Health;
    ATTRIBUTE_ACCESSORS(UMyAttributeSet, Health)       // 生成 GetHealthAttribute()/GetHealth(

    UPROPERTY(BlueprintReadOnly, Category = "Vital", ReplicatedUsing = OnRep_MaxHealth)
    FGameplayAttributeData MaxHealth;
    ATTRIBUTE_ACCESSORS(UMyAttributeSet, MaxHealth)

    UPROPERTY(BlueprintReadOnly, Category = "Vital", ReplicatedUsing = OnRep_Mana)
    FGameplayAttributeData Mana;
    ATTRIBUTE_ACCESSORS(UMyAttributeSet, Mana)

    UPROPERTY(BlueprintReadOnly, Category = "Combat", ReplicatedUsing = OnRep_Armor)
    FGameplayAttributeData Armor;
    ATTRIBUTE_ACCESSORS(UMyAttributeSet, Armor)

    // ——— Meta 属性：不复制，只在服务器上作为"中转站"使用 ———
    UPROPERTY(BlueprintReadOnly, Category = "Meta", meta = (HideFromLevelInfos))
    FGameplayAttributeData IncomingDamage;
    ATTRIBUTE_ACCESSORS(UMyAttributeSet, IncomingDamage)

    virtual void GetLifetimeReplicatedProps(TArray<FLifetimeProperty>& OutLifetimeProps) const
    virtual void PreAttributeChange(const FGameplayAttribute& Attribute, float& NewValue) over
    virtual void PostAttributeChange(const FGameplayAttribute& Attribute, float OldValue, floa
    virtual bool PreGameplayEffectExecute(FGameplayEffectModCallbackData& Data) override;
    virtual void PostGameplayEffectExecute(const FGameplayEffectModCallbackData& Data) overric
```

```
protected:
    UFUNCTION() void OnRep_Health(const FGameplayAttributeData& OldValue);
    UFUNCTION() void OnRep_MaxHealth(const FGameplayAttributeData& OldValue);
    UFUNCTION() void OnRep_Mana(const FGameplayAttributeData& OldValue);
    UFUNCTION() void OnRep_Armor(const FGameplayAttributeData& OldValue);
};
```

MyAttributeSet.cpp —— 复制部分

```
void UMyAttributeSet::GetLifetimeReplicatedProps(TArray<FLifetimeProperty>& OutLifetimeProps)
{
    Super::GetLifetimeReplicatedProps(OutLifetimeProps);
    // REPNOTIFY_Always：即使新值与客户端本地值相同也要触发 OnRep（预测需要，见 13.1）
    DOREPLIFETIME_CONDITION_NOTIFY(UMyAttributeSet, Health,    COND_None,      REPNOTIFY_Alway
    DOREPLIFETIME_CONDITION_NOTIFY(UMyAttributeSet, MaxHealth, COND_None,      REPNOTIFY_Alway
    DOREPLIFETIME_CONDITION_NOTIFY(UMyAttributeSet, Mana,      COND_OwnerOnly, REPNOTIFY_Alway
    DOREPLIFETIME_CONDITION_NOTIFY(UMyAttributeSet, Armor,     COND_None,      REPNOTIFY_Alway
}

void UMyAttributeSet::OnRep_Health(const FGameplayAttributeData& OldValue)
{
    // 通知 ASC：把服务器下发的值同步进聚合器，并广播属性变化委托
    GAMEPLAYATTRIBUTE_REPNOTIFY(UMyAttributeSet, Health, OldValue);
}
// 其余 OnRep_* 写法相同
```

5.3 回调的调用顺序

AttributeSet 提供了一组虚函数，分别在数值变化的不同阶段被调用。**弄清楚谁在什么时候被调用**，是避免数值 Bug 的关键：

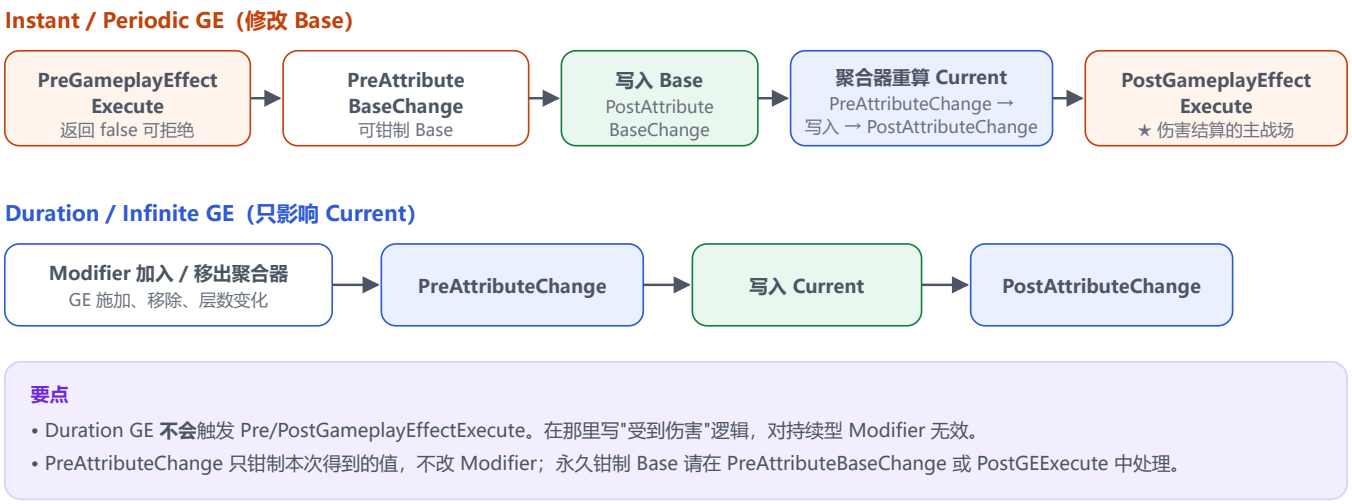


图 5-2 AttributeSet 回调顺序。Instant 类 GE 走上面一条（多两个 Execute 回调），持续类 GE 走下面一条

5.4 钳制与 Meta 属性：伤害的标准写法

为什么不让伤害 GE 直接 `Health -= X`？因为扣血之前往往还有很多步骤：护盾吸收、伤害反弹、吸血、击杀判定、伤害数字显示。**Meta 属性**（例如 `IncomingDamage`）就是一个临时中转站：GE 只往它里面写数值，在 `PostGameplayEffectExecute` 中统一处理后清零。

MyAttributeSet.cpp —— 钳制与伤害结算

```cpp
void UMyAttributeSet::PreAttributeChange(const FGameplayAttribute& Attribute, float& NewValue)
{
    Super::PreAttributeChange(Attribute, NewValue);
    if (Attribute == GetHealthAttribute())
    {
        NewValue = FMath::Clamp(NewValue, 0.f, GetMaxHealth());
    }
}

void UMyAttributeSet::PostAttributeChange(const FGameplayAttribute& Attribute, float OldValue,
{
    Super::PostAttributeChange(Attribute, OldValue, NewValue);
    // MaxHealth 变化时，按比例调整当前血量（例如 500/1000 → 加装备后 600/1200）
    if (Attribute == GetMaxHealthAttribute() && OldValue > 0.f)
    {
        UAbilitySystemComponent* ASC = GetOwningAbilitySystemComponent();
        if (!ASC || !ASC->IsOwnerActorAuthoritative()) return;   // 只在服务器修正，客户端等复制
        const float NewHealth = GetHealth() * NewValue / OldValue;
        ASC->ApplyModToAttribute(GetHealthAttribute(), EGameplayModOp::Override, NewHealth);
    }
}

bool UMyAttributeSet::PreGameplayEffectExecute(FGameplayEffectModCallbackData& Data)
{
    // 无敌：直接拒绝本次伤害执行
    if (Data.EvaluatedData.Attribute == GetIncomingDamageAttribute()
        && Data.Target.HasMatchingGameplayTag(MyTags::State_Buff_Invulnerable))
    {
        return false;
    }
    return true;
}

void UMyAttributeSet::PostGameplayEffectExecute(const FGameplayEffectModCallbackData& Data)
{
    Super::PostGameplayEffectExecute(Data);

    if (Data.EvaluatedData.Attribute == GetIncomingDamageAttribute())
    {
        const float Damage = GetIncomingDamage();
        SetIncomingDamage(0.f);                          // Meta 属性用完立即清零
        if (Damage ≤ 0.f) return;
```

```
        const float NewHealth = FMath::Clamp(GetHealth() - Damage, 0.f, GetMaxHealth());
        SetHealth(NewHealth);

        // 来源信息都在 Context 里：谁打的、用什么打的、命中点
        const FGameplayEffectContextHandle& Ctx = Data.EffectSpec.GetContext();
        AActor* Instigator = Ctx.GetOriginalInstigator();

        if (NewHealth ≤ 0.f && !Data.Target.HasMatchingGameplayTag(MyTags::State_Dead))
        {
            // 通知"死亡"：发 GameplayEvent，由死亡技能去处理（掉落、计分、复活计时）
            FGameplayEventData Payload;
            Payload.Instigator = Instigator;
            Payload.EventMagnitude = Damage;
            Data.Target.HandleGameplayEvent(MyTags::Event_Death, &Payload);
        }
    }
}
```

🔬 **深入源码：SetHealth 做了什么**

宏生成的 `SetHealth(v)` 内部调用的是 `ASC→SetNumericAttributeBase(Attr, v)`，也就是**修改 Base 并让聚合器重新计算 Current**，而不是直接改 float 字段。所以在 AttributeSet 外部，请始终使用这些 Setter 或 GE 来修改属性，不要直接给 `Health.SetCurrentValue()` 赋值，否则聚合器状态会被绕过。

5.5 监听属性变化（UI）

血条控件监听 Health

```
ASC→GetGameplayAttributeValueChangeDelegate(UMyAttributeSet::GetHealthAttribute())
   .AddUObject(this, &UMyHealthBar::OnHealthChanged);

void UMyHealthBar::OnHealthChanged(const FOnAttributeChangeData& Data)
{
    SetPercent(Data.NewValue / CachedMaxHealth);
    // 注意：Data.GEModData 只在服务器上有效（客户端收到的是属性复制，没有 GE 上下文）
}
```

蓝图中可以直接使用异步节点 **Wait for Attribute Changed**（`UAbilityAsync_WaitAttributeChanged`），不需要写 C++。

5.6 派生属性

"最大生命值 = 体质 × 10 + 200"这种派生关系，**不要在代码里手动同步**。做法是给角色挂一个 **Infinite GE**，其中 MaxHealth 的 Modifier 使用 *Attribute Based* 类型的数值，捕获自身的 Vitality，并且**不勾选 Snapshot**。之后 Vitality 发生变化时，聚合器会自动重新计算 MaxHealth。

5.7 多个 AttributeSet

一个 ASC 可以有多个 AttributeSet，例如 Lyra 把属性拆成 `HealthSet` 和 `CombatSet`。但**同一个类的 AttributeSet 只能有一个**，ASC 按类查找属性集，有重复时只会返回第一个。

本章小结

- **Instant 改 Base，Duration / Infinite 改 Current。**Base 不被持续型效果修改，因此撤销总是精确的。
- 回调顺序：PreGEExecute → PreBaseChange → Base → PreAttributeChange → Current → PostAttributeChange → PostGEExecute；后两个 Execute 回调只在 Instant / Periodic 时触发。
- 伤害走 **Meta 属性** IncomingDamage，在 PostGameplayEffectExecute 中统一结算后清零。
- 复制使用 `REPNOTIFY_Always` + `GAMEPLAYATTRIBUTE_REPNOTIFY`，原因在第 13 章解释。

CHAPTER 06

GameplayEffect（上）：时长、修饰器与聚合公式

GE 是 GAS 中"改变世界"的唯一正规途径。本章讲 GE 的结构，以及多个 Modifier 叠加时的计算方式。

6.1 GE 的结构

GE 是一个**纯数据**类：一般在蓝图中继承 `UGameplayEffect` 并配置字段，**不写逻辑**。需要逻辑的部分（复杂公式、条件判断）通过插件类挂进来，例如 MMC、Execution 和 GEComponent。

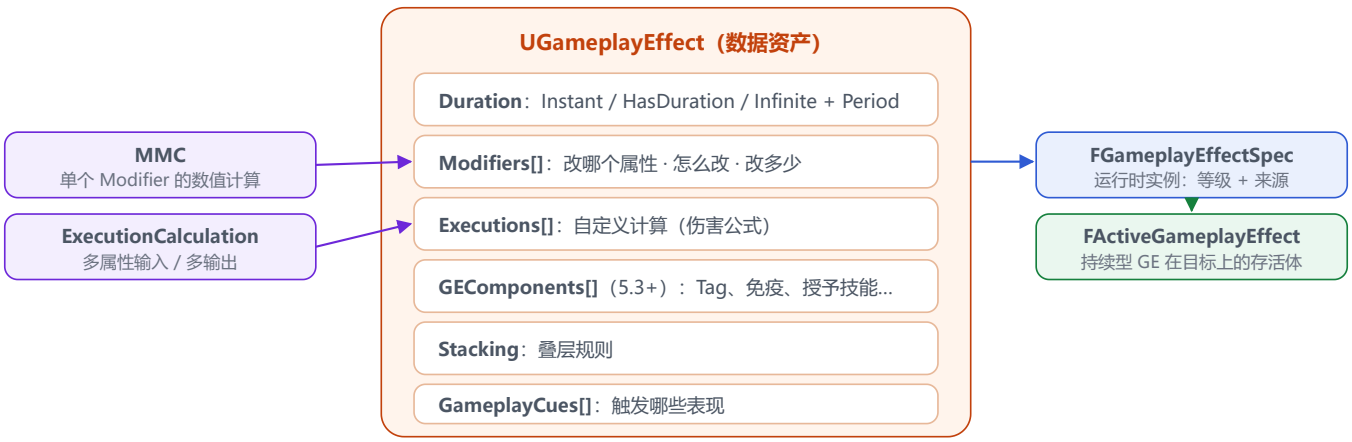


图 6-1 GE 的结构：左侧是可插入的计算类，右侧是它在运行时的两种形态

6.2 三种时长策略

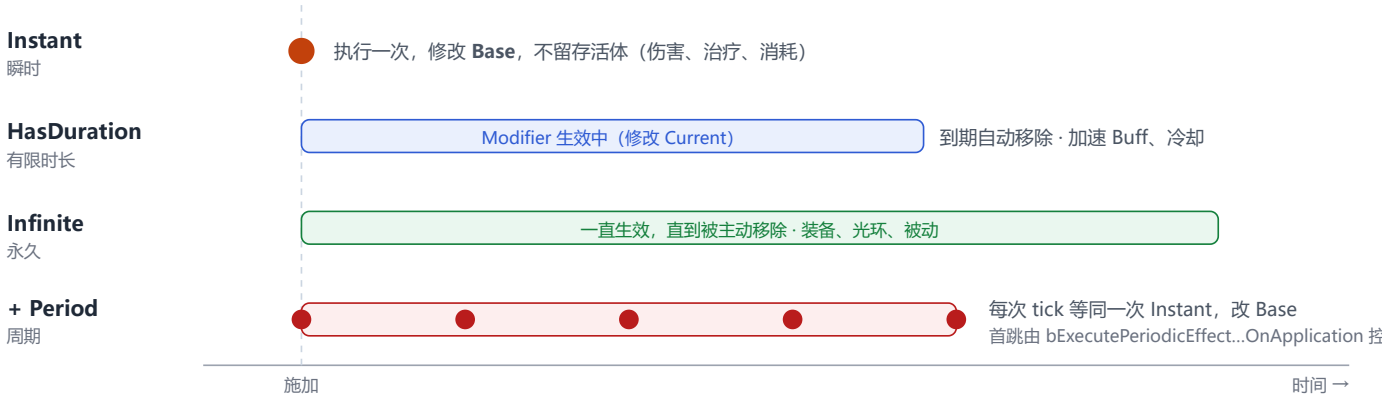


图 6-2 时长策略。带 Period 的持续型 GE 中，Modifier 不再常驻聚合器，而是在每次 tick 时作为 Instant 执行

⚠️ 坑点：DOT 加速 Buff 的区别

一个 HasDuration GE **没有 Period** 时，Modifier 修改 Current，到期后自动撤销（例如加速）；**有 Period** 时，Modifier 在每次 tick 时永久修改 Base，到期后不会撤销（例如燃烧掉的血）。"中毒 5 秒、每秒 −10 血"必须配置 Period = 1。

🔬 深入源码：Period 与 Ongoing Tag Requirements

持续型 GE 可以配置"持续期间需要满足的 Tag 条件"（5.3+ 使用 `TargetTagRequirementsGameplayEffectComponent` 的 Ongoing 部分）。条件不满足时 GE 会被**抑制（Inhibited）**：Modifier 暂时从聚合器中移除，GE 本身仍然存在。周期效果被抑制后重新激活时，按 `PeriodicInhibitionPolicy`（NeverReset / ResetPeriod / ExecuteAndResetPeriod）决定计时方式。

6.3 Modifier 的四个要素

字段	含义
Attribute	要改的属性。属性下拉列表来自所有 AttributeSet；属性加 `meta=(HideFromModifiers)` 可以从列表中隐藏
Modifier Op	运算方式：AddBase / MultiplyAdditive / DivideAdditive / MultiplyCompound / AddFinal / Override
Magnitude	数值来源，有四种计算方式（见 6.4）
Source / Target Tags	Modifier 级别的条件。例如"仅当目标处于燃烧状态时 +30% 伤害"，条件不满足时该 Modifier 不参与聚合

6.4 数值（Magnitude）的四种计算方式

类型	原理	典型用途
Scalable Float	常数 × 曲线表[Spec 等级]	"1 级 50 伤害，5 级 150 伤害"
Attribute Based	`Coefficient × (Attr + PreMultiplyAdditive) + PostMultiplyAdditive`，Attr 可以取 Source 或 Target 的属性，可选 Current / Base / Bonus（Current − Base）	"治疗量 = 施法者法强 × 0.6"、派生属性
Custom Calculation Class (MMC)	继承 `UGameplayModMagnitudeCalculation`，在 C++ / 蓝图中计算一个 float	依赖多个属性或 Tag 的单值计算，例如受冷却缩减影响的冷却时长
Set By Caller	施加前由代码用 Tag 作为键写入 Spec	由技能逻辑决定的数值：蓄力伤害、按距离衰减

🔬 **深入源码：Snapshot（快照）是什么**

捕获属性时可以选择是否快照。**Source 快照**在 Spec *创建*时记录施法者的属性值，之后施法者属性变化不影响已发出的效果（例如火球已经飞出去，再加攻击力也不影响这颗火球）。**不快照**时，持续型 GE 会实时跟随属性变化（例如光环随施法者属性变化）。Target 属性通常在 GE *施加*时才捕获。

6.5 聚合公式：多个 Modifier 怎么叠加

同一个属性上可能同时存在十几个 Modifier。GAS 先把**同类运算**汇总，再按固定顺序计算：

$$Current = ((Base + \Sigma AddBase) \times MulAdd \div DivAdd \times \Pi MulCompound) + \Sigma AddFinal$$

（存在生效的 Override 时，直接返回 Override 的值）



两种"乘"的区别（最容易搞错的地方）

- **MultiplyAdditive**（老版本叫 Multiplicitive）：先把 (Mi − 1) 相加再加 1。+20% 和 +30% 合起来是 ×1.5，**不是** 1.2 × 1.3 = 1.56。适合"同类加成相加"的设计（大多数游戏中"增加 X% 攻击"都是这个语义）。DivideAdditive 同理：÷2 和 ÷2 合起来是 ÷3。
- **MultiplyCompound**：逐个连乘，1.5 × 1.5 = 2.25。适合"独立乘区"，例如"最终伤害 ×1.5"。

图 6-3 聚合流水线示例：攻击力 Base 100 在五类 Modifier 作用下变为 280

🧪 交互实验：聚合器计算器

在每类中填入多个数值，用逗号分隔。MultiplyAdditive / Compound / Divide 填倍率（例如 1.2 表示 +20%）。

Base 100　AddBase 20　MultiplyAdditive 1.2,1.3

DivideAdditive　　MultiplyCompound 1.5　AddFinal 10

Override 留空

```
① Base + ΣAddBase          = 100 + 20 = 120
② × MultiplyAdditive        = 1 + Σ(Mi-1) = 1.5   → 180
③ ÷ DivideAdditive          = 1 + Σ(Di-1) = 1     → 180
④ × ΠMultiplyCompound       = 1.5    → 270
⑤ + ΣAddFinal               = 10     → CurrentValue = 280

对比：若把 MultiplyAdditive 改成连乘，结果是 290.8
```

🕑 版本差异

老版本只有 `Additive / Multiplicitive / Division / Override` 四种，公式是 `((Base + Additive) × Multiplicitive) ÷ Division`。新版本把它们改名为 AddBase / MultiplyAdditive / DivideAdditive（枚举值不变，老资产无需迁移），并新增了 `MultiplyCompound` 和 `AddFinal`。在老版本中要实现"独立乘区"，需要借助 Evaluation Channel 或者 Execution。

⚠️ 坑点：Override 和多个 Override

Override 会让其他 Modifier 全部失效，而且多个 Override 同时存在时，结果取决于聚合器中 Modifier 的顺序。除了"初始化属性"这种一次性的 Instant 用法，**尽量不要在持续型 GE 上使用 Override**。"定身（速度为 0）"应该用 MultiplyCompound ×0，或者授予一个让移动组件读取的 Tag。

本章小结

- GE = Duration + Modifiers + Executions + Components + Stacking + Cues，是纯数据。
- Instant 和 Period tick 修改 Base；不带 Period 的持续型 GE 只往聚合器中添加 Modifier。
- 聚合顺序：**AddBase** → **MultiplyAdditive** → **DivideAdditive** → **MultiplyCompound** → **AddFinal**，"Additive" 类乘法是先把增量相加。

CHAPTER 07

GameplayEffect（下）：Spec、计算类、堆叠与组件

把 GE 用到实际项目中：如何传入运行时数值、如何写伤害公式、如何处理叠层，以及 5.3 之后的组件化配置。

7.1 Spec 与 Context：一次施加带的信息

在技能中构造并施加 GE

```
// 1) 由施法者 ASC 生成 Spec（此时 Source 侧快照属性被捕获）
FGameplayEffectContextHandle Ctx = SourceASC->MakeEffectContext();
Ctx.AddSourceObject(this);              // 来源对象（技能 / 武器）
Ctx.AddHitResult(Hit);                  // 命中信息，Cue 可以读到命中点
FGameplayEffectSpecHandle Spec = SourceASC->MakeOutgoingSpec(GE_Damage, AbilityLevel, Ctx);

// 2) 写入 SetByCaller 数值（Tag 作为键）
Spec.Data->SetSetByCallerMagnitude(MyTags::SetByCaller_Damage, 120.f);
// 也可以追加动态 AssetTag，让 Execution 识别伤害类型
Spec.Data->AddDynamicAssetTag(MyTags::Damage_Type_Magic);

// 3) 施加到目标（返回句柄：Instant GE 的句柄无效，持续型 GE 可用于之后移除）
FActiveGameplayEffectHandle H = SourceASC->ApplyGameplayEffectSpecToTarget(*Spec.Data.Get(), T
```

在 GameplayAbility 内部，使用封装好的 `MakeOutgoingGameplayEffectSpec()` 和 `ApplyGameplayEffectSpecToTarget()` 更方便，它们会自动填好技能等级，并**带上当前的预测键**（第 12 章会讲为什么这一点很重要）。

✅ **实践建议：自定义 Context**

需要在 GE 上携带更多信息（是否暴击、技能连段数、投射物 ID）时，继承 `FGameplayEffectContext`，重写 `GetScriptStruct`、`Duplicate` 和 `NetSerialize`，再在自定义的 `UAbilitySystemGlobals::AllocGameplayEffectContext()` 中返回它，并在 `DefaultGame.ini` 中配置 `AbilitySystemGlobalsClassName`。Context 中的对象指针**不会阻止 GC**，请使用弱引用。

7.2 MMC：受冷却缩减影响的冷却时长

MMC_Cooldown.cpp

```
UMMC_Cooldown::UMMC_Cooldown()
{
    CDRDef.AttributeToCapture = UMyAttributeSet::GetCooldownReductionAttribute();
    CDRDef.AttributeSource    = EGameplayEffectAttributeCaptureSource::Source;
    CDRDef.bSnapshot          = true;          // 进入冷却时的冷却缩减值为准
    RelevantAttributesToCapture.Add(CDRDef);
}

float UMMC_Cooldown::CalculateBaseMagnitude_Implementation(const FGameplayEffectSpec& Spec) co
{
    FAggregatorEvaluateParameters Params;
    Params.SourceTags = Spec.CapturedSourceTags.GetAggregatedTags();
    Params.TargetTags = Spec.CapturedTargetTags.GetAggregatedTags();

    float CDR = 0.f;
    GetCapturedAttributeMagnitude(CDRDef, Spec, Params, CDR);
    CDR = FMath::Clamp(CDR, 0.f, 0.4f);         // 冷却缩减上限 40%

    // 基础冷却由技能通过 SetByCaller 传入，这样所有技能可以共用同一个冷却 GE
    const float BaseCD = Spec.GetSetByCallerMagnitude(MyTags::SetByCaller_Cooldown, false, 1.f
    return BaseCD * (1.f - CDR);
}
```

7.3 ExecutionCalculation：伤害公式

当一个计算需要**读取多个属性（Source 和 Target 都有）**，或者需要**输出到多个属性**时，就该使用 Execution。它是整个项目中伤害规则的唯一实现位置：

ExecCalc_Damage.cpp

```cpp
struct FDamageStatics
{
    DECLARE_ATTRIBUTE_CAPTUREDEF(AttackPower);
    DECLARE_ATTRIBUTE_CAPTUREDEF(ArmorPenetration);
    DECLARE_ATTRIBUTE_CAPTUREDEF(CritChance);
    DECLARE_ATTRIBUTE_CAPTUREDEF(Armor);
    FDamageStatics()
    {   //                          类                  属性                  来源      快照
        DEFINE_ATTRIBUTE_CAPTUREDEF(UMyAttributeSet, AttackPower,      Source, true);
        DEFINE_ATTRIBUTE_CAPTUREDEF(UMyAttributeSet, ArmorPenetration, Source, true);
        DEFINE_ATTRIBUTE_CAPTUREDEF(UMyAttributeSet, CritChance,       Source, true);
        DEFINE_ATTRIBUTE_CAPTUREDEF(UMyAttributeSet, Armor,            Target, false);
    }
};
static const FDamageStatics& DamageStatics() { static FDamageStatics S; return S; }

UExecCalc_Damage::UExecCalc_Damage()
{
    RelevantAttributesToCapture.Add(DamageStatics().AttackPowerDef);
    RelevantAttributesToCapture.Add(DamageStatics().ArmorPenetrationDef);
    RelevantAttributesToCapture.Add(DamageStatics().CritChanceDef);
    RelevantAttributesToCapture.Add(DamageStatics().ArmorDef);
}

void UExecCalc_Damage::Execute_Implementation(const FGameplayEffectCustomExecutionParameters&
                                              FGameplayEffectCustomExecutionOutput& Out) const
{
    const FGameplayEffectSpec& Spec = ExecParams.GetOwningSpec();
    FAggregatorEvaluateParameters P;
    P.SourceTags = Spec.CapturedSourceTags.GetAggregatedTags();
    P.TargetTags = Spec.CapturedTargetTags.GetAggregatedTags();

    float AP = 0, Pen = 0, Crit = 0, Armor = 0;
    ExecParams.AttemptCalculateCapturedAttributeMagnitude(DamageStatics().AttackPowerDef,
    ExecParams.AttemptCalculateCapturedAttributeMagnitude(DamageStatics().ArmorPenetrationDef,
    ExecParams.AttemptCalculateCapturedAttributeMagnitude(DamageStatics().CritChanceDef,
    ExecParams.AttemptCalculateCapturedAttributeMagnitude(DamageStatics().ArmorDef,

    // 技能基础伤害 + 攻击力加成系数，都由技能通过 SetByCaller 传入
    const float BaseDmg = Spec.GetSetByCallerMagnitude(MyTags::SetByCaller_Damage, false, 0.f)
    const float Coef    = Spec.GetSetByCallerMagnitude(MyTags::SetByCaller_AttackCoef, false,
    float Damage = BaseDmg + AP * Coef;
```

```
    // 护甲减伤：有效护甲 = 护甲 × (1 − 穿透)，减伤乘数 = 100 / (100 + 有效护甲)
    const float EffArmor = FMath::Max(0.f, Armor * (1.f - FMath::Clamp(Pen, 0.f, 1.f)));
    Damage *= 100.f / (100.f + EffArmor);

    // 暴击（Execution 只在服务器运行，随机数是权威结果）
    if (FMath::FRand() < Crit) { Damage *= 2.f; /* 可以写入自定义 Context，供 UI 显示暴击 */ }

    if (Damage > 0.f)
    {   // 输出到 Meta 属性，由 AttributeSet::PostGameplayEffectExecute 结算（新版本中 Additive
        Out.AddOutputModifier(FGameplayModifierEvaluatedData(
            UMyAttributeSet::GetIncomingDamageAttribute(), EGameplayModOp::Additive, Damage));
    }
}
```

> 🌐 **网络视角：Execution 不参与预测**
>
> Execution 只能用于 Instant 和 Periodic GE，并且在**服务器**上计算（本地预测的技能也不会在客户端执行 Execution）。这对伤害结算正合适：随机数、护甲读取都以服务器为准，客户端无法作弊。代价是伤害数字需要等服务器结果，无法预测显示。

	Modifier + Magnitude	MMC	Execution
输入	常数 / 单个属性	多个属性 + Tag + SetByCaller	多个属性 + Tag + SetByCaller
输出	1 个属性	1 个数值	**多个属性**
可用于持续型 GE	**是**	**是（可随属性实时变化）**	**否（仅 Instant / Periodic）**
可预测	**是**	**是**	**否**
典型用途	Buff、消耗	冷却时长、按等级缩放	伤害、治疗、吸血

7.4 Stacking：叠层

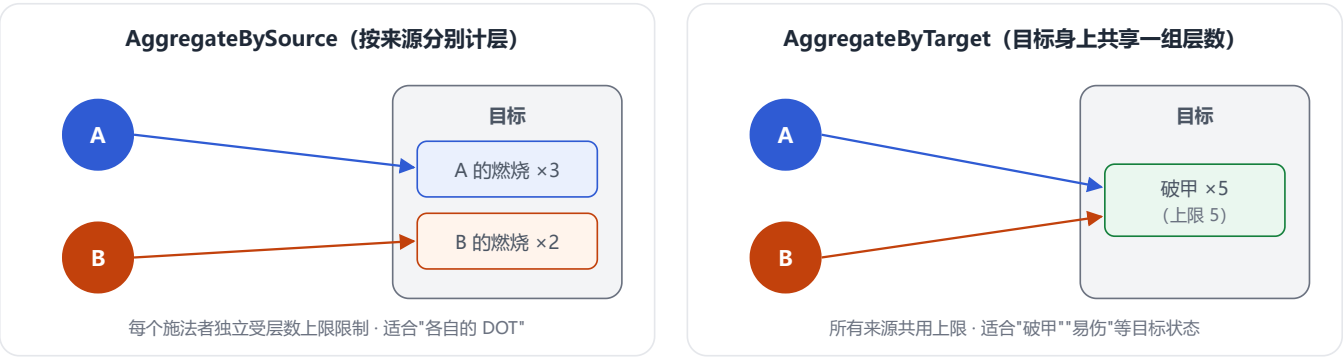


图 7-1 两种堆叠类型。层数存在 `FActiveGameplayEffect` 的 StackCount 中，Modifier 的数值会乘以层数

设置	可选值	说明
Stacking Type	None / AggregateBySource / AggregateByTarget	None 表示每次施加都创建一个独立的 ActiveGE
Stack Limit Count	整数	层数上限。超过上限时可以触发 Overflow GE（例如"叠满 5 层引爆"）
Stack Duration Refresh Policy	RefreshOnSuccessfulApplication / NeverRefresh	新叠一层时是否重置持续时间
Stack Period Reset Policy	ResetOnSuccessfulApplication / NeverReset	新叠一层时是否重置周期计时（避免频繁命中导致 DOT 永远不跳）
Stack Expiration Policy	ClearEntireStack / RemoveSingleStackAndRefreshDuration / RefreshDuration	到期时全部清除 / 每次掉一层 / 永不过期（仅刷新）

7.5 GE 组件化（UE 5.3+）

5.3 之前，GE 面板上有一大堆 Tag 相关字段，大部分 GE 根本用不上。5.3 把它们拆成了**按需添加的组件**：

组件	作用	替代的老字段
`UTargetTagsGameplayEffectComponent`	GE 生效期间给目标授予 Tag	GrantedTags（InheritableOwnedTagsContainer）
`UAssetTagsGameplayEffectComponent`	GE 自身的 Tag，用于查询、驱散、免疫判定	GameplayEffectAssetTag
`UBlockAbilityTagsGameplayEffectComponent`	生效期间阻止带指定 Tag 的技能激活	GrantedBlockedAbilityTags
`UTargetTagRequirementsGameplayEffectComponent`	施加条件 / 持续条件 / 移除条件	Application / Ongoing / Removal TagRequirements
`URemoveOtherGameplayEffectComponent`	施加时移除其他符合条件的 GE（驱散）	RemoveGameplayEffectsWithTags
`UImmunityGameplayEffectComponent`	生效期间对符合查询条件的 GE 免疫	GrantedApplicationImmunityTags
`UAbilitiesGameplayEffectComponent`	生效期间授予技能，结束时收回	GrantedAbilities
`UAdditionalEffectsGameplayEffectComponent`	施加或结束时连带施加其他 GE	ConditionalGameplayEffects 等
`UChanceToApplyGameplayEffectComponent`	按概率施加	ChanceToApplyToTarget
`UCustomCanApplyGameplayEffectComponent`	自定义是否可以施加	ApplicationRequirements
`UGameplayEffectUIData` 系列	UI 显示数据（图标、描述）	UIData

C++ 中给 GE 添加组件（构造函数中）

```cpp
UGE_Stun::UGE_Stun()
{
    DurationPolicy = EGameplayEffectDurationType::HasDuration;
    DurationMagnitude = FScalableFloat(1.0f);

    // 授予 State.Debuff.Stun
    UTargetTagsGameplayEffectComponent* TagsComp = CreateDefaultSubobject<UTargetTagsGameplayE
    GEComponents.Add(TagsComp);
    FInheritedTagContainer Granted = TagsComp->GetConfiguredTargetTagChanges();
    Granted.AddTag(MyTags::State_Debuff_Stun);
    TagsComp->SetAndApplyTargetTagChanges(Granted);
}
```

✅ 实践建议：自定义 GEComponent

项目特有的 GE 行为（例如"生效期间显示描边""施加时记录战斗日志"）可以继承 `UGameplayEffectComponent`，重写 `CanGameplayEffectApply`、`OnActiveGameplayEffectAdded`、`OnGameplayEffectApplied` 等钩子来实现，不需要修改引擎。这正是 5.3 组件化想要解决的扩展问题。

7.6 一次施加的完整流水线

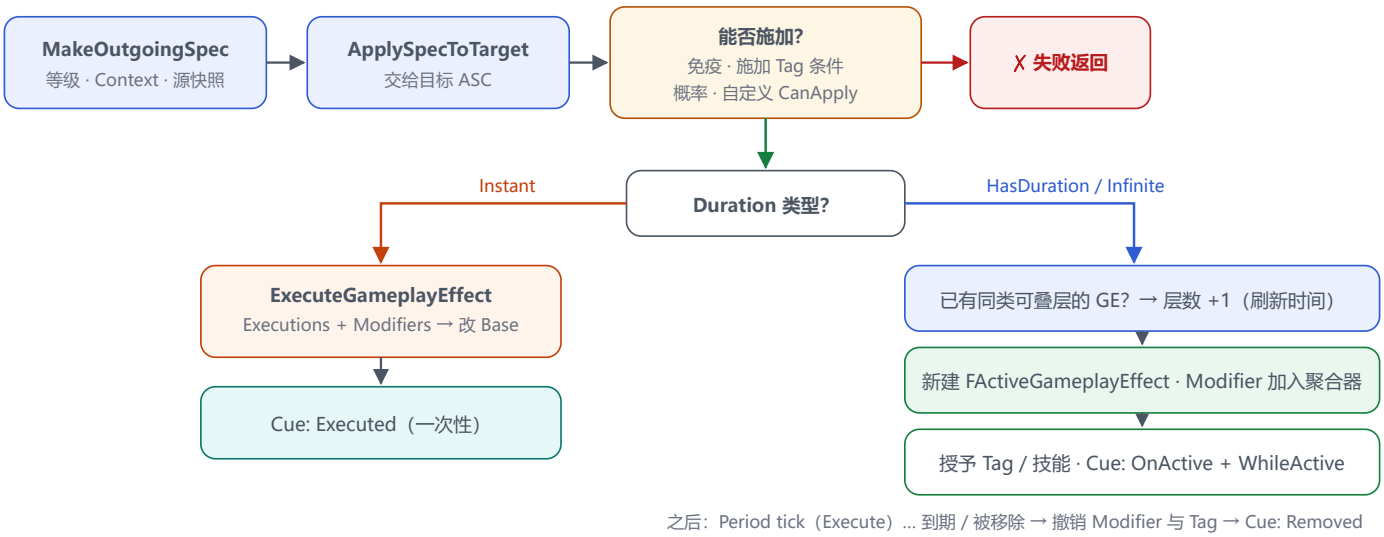


图 7-2 GE 从创建 Spec 到在目标身上生效的流水线

7.7 查询、移除与监听

常用的查询与移除操作

```
// 按 GE 自身的 AssetTag 驱散所有控制效果
FGameplayEffectQuery Q = FGameplayEffectQuery::MakeQuery_MatchAnyEffectTags(FGameplayTagContai
ASC->RemoveActiveEffects(Q);

// 查询某个冷却的剩余时间（UI 转圈）
TArray<TPair<float, float>> Remain = ASC->GetActiveEffectsTimeRemainingAndDuration(
    FGameplayEffectQuery::MakeQuery_MatchAnyOwningTags(FGameplayTagContainer(MyTags::Cooldown_

// 监听：自己被施加了任何持续型 GE（服务器和拥有者客户端都会触发）
ASC->OnActiveGameplayEffectAddedDelegateToSelf.AddUObject(this, &ThisClass::OnEffectAdded);
// 监听某个 GE 的层数变化
ASC->OnGameplayEffectStackChangeDelegate(Handle)->AddUObject(this, &ThisClass::OnStackChanged)
```

⚠️ 坑点：在客户端移除 GE

GE 的移除**只能由服务器发起**，并且**不能被预测**。在客户端调用 `RemoveActiveGameplayEffect`，下一次复制时 GE 又会出现。

本章小结

- Spec 携带等级、Context 和 SetByCaller；在技能内施加 GE 时使用技能自带的接口，会自动带上预测键。
- 单值计算用 MMC，多输入多输出（伤害）用 Execution。Execution 只在服务器运行，不参与预测。
- Stacking 分为按来源计层和按目标计层，配合刷新策略和过期策略可以实现各种层数机制。
- 5.3+ 的 Tag、免疫、授予技能等功能都通过 **GEComponents** 配置，项目特有的行为可以自定义组件实现。

CHAPTER 08

GameplayAbility：技能的生命周期

GE 是"改什么"，GA 是"做什么"。本章讲 GA 从授予到结束的完整生命周期、Tag 规则、消耗与冷却，以及决定联网行为的四个策略。

8.1 生命周期全图

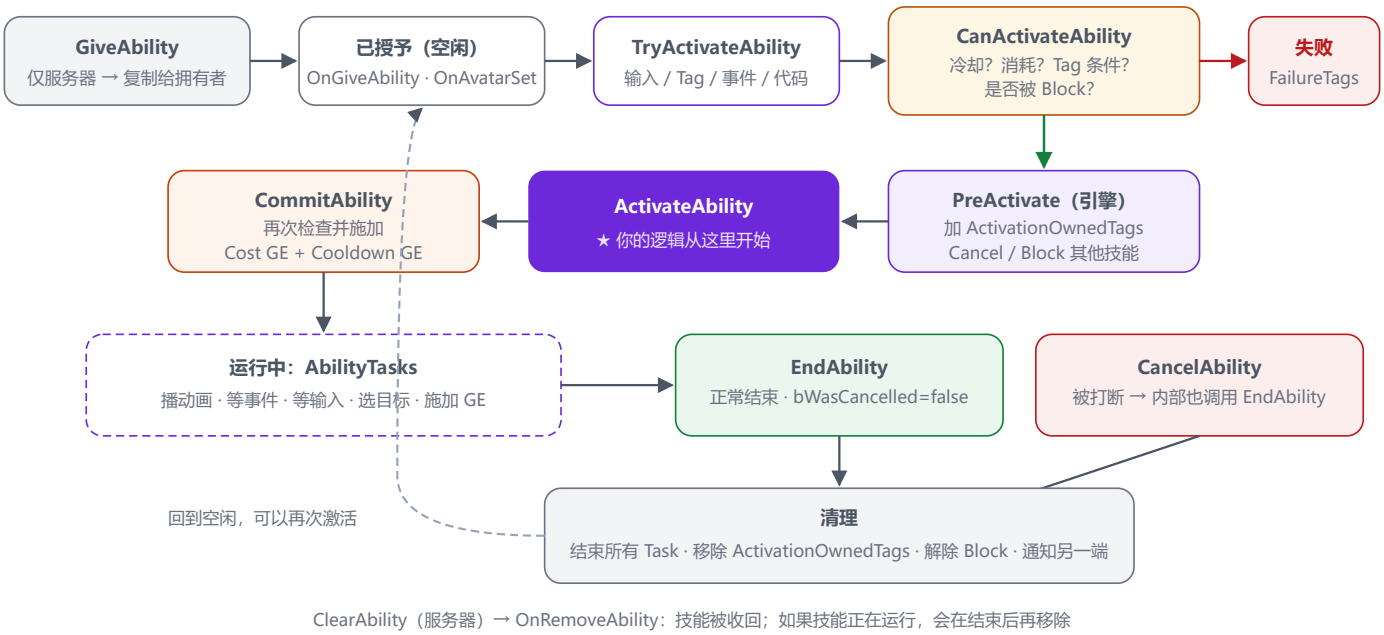


图 8-1 GameplayAbility 的生命周期。紫色是技能执行部分，橙色是资源结算部分

> ⚠️ **坑点：忘记 EndAbility**
>
> 技能不会自动结束。如果某个分支（例如 Montage 被打断的回调）没有调用 `EndAbility`，技能会一直处于激活状态：`ActivationOwnedTags` 不会移除，InstancedPerActor 技能也无法再次激活，表现为"技能按了没反应"。**每个 Task 的每个出口回调（Completed / Interrupted / Cancelled）都要检查是否需要结束技能。**

8.2 Tag 规则：十个字段

GA 上有十个 Tag 容器，组合起来可以描述几乎所有"技能之间能否并存"的规则：

字段	作用对象	含义	例子（冲锋斩）
AssetTags（老名 AbilityTags）	本技能	技能自己的身份标签	`Ability.Hero.Q`、`Ability.Type.Dash`
CancelAbilitiesWithTag	其他技能	激活时**打断**正在运行、带这些 Tag 的技能	`Ability.Type.Channel`（打断自己的引导技能）
BlockAbilitiesWithTag	其他技能	运行期间**阻止**带这些 Tag 的技能激活	`Ability.Type.Attack`（冲刺中不能普攻）
ActivationOwnedTags	拥有者	运行期间给自己加上这些 Tag	`State.Dashing`
ActivationRequiredTags	拥有者	拥有者**必须有**这些 Tag 才能激活	（变身后才能用的技能填 `State.Transformed`）
ActivationBlockedTags	拥有者	拥有者有这些 Tag 时**不能**激活	`State.Debuff.Stun`、`State.Debuff.Root`、`State.Dead`
Source/Target Required/Blocked	来源 / 目标	由事件或 TargetData 激活时，检查来源和目标的 Tag	（"只能对敌方英雄释放"）

✅ **实践建议：Block 与 ActivationBlockedTags 的区别**

`BlockAbilitiesWithTag` 是**技能之间**的互斥（通过 ASC 的 BlockedAbilityTags 计数实现）；`ActivationBlockedTags` 检查的是**角色状态**。"眩晕时所有技能都不能放"有两种做法：①所有技能的 ActivationBlockedTags 都填 Stun；②眩晕 GE 加一个 `BlockAbilityTagsGameplayEffectComponent`，阻止 `Ability` 根 Tag 下的所有技能。②只需改一处，更好维护。别忘了还要打断**正在运行**的技能（见 4.4 的 Tag 监听示例）。

8.3 消耗与冷却

在 GA 上配置 `CostGameplayEffectClass` 和 `CooldownGameplayEffectClass`：

- **Cost GE**：Instant，Modifier 为 `Mana AddBase -30`。`CheckCost` 会模拟施加，判断属性是否会小于 0。
- **Cooldown GE**：HasDuration，**必须**通过 TargetTags 组件授予一个冷却 Tag（例如 `Cooldown.Hero.Q`）。`CheckCooldown` 只检查拥有者身上是否有这个 Tag。
- 在 `ActivateAbility` 中调用 `CommitAbility()`，**并检查返回值**：激活之后，资源可能已经被其他效果改变。

默认做法要求每个技能各配一个冷却 GE，技能多了很难维护。常见的做法是**所有技能共用一个冷却 GE**，冷却时长和冷却 Tag 在运行时注入：

MyGameplayAbility.cpp —— 通用冷却

```
// 头文件中：
// UPROPERTY(EditDefaultsOnly, Category="Cooldown") FScalableFloat CooldownDuration;   // 按技
// UPROPERTY(EditDefaultsOnly, Category="Cooldown") FGameplayTagContainer CooldownTags; // 例如
// UPROPERTY(Transient) FGameplayTagContainer TempCooldownTags;

const FGameplayTagContainer* UMyGameplayAbility::GetCooldownTags() const
{
    // CheckCooldown 用它来判断是否处于冷却中
    FGameplayTagContainer* Mutable = const_cast<FGameplayTagContainer*>(&TempCooldownTags);
    Mutable->Reset();
    if (const FGameplayTagContainer* Parent = Super::GetCooldownTags()) { Mutable->AppendTags(
    Mutable->AppendTags(CooldownTags);
    return Mutable;
}

void UMyGameplayAbility::ApplyCooldown(const FGameplayAbilitySpecHandle Handle, const FGamepla
                                       const FGameplayAbilityActivationInfo ActivationInfo) co
{
    if (UGameplayEffect* CooldownGE = GetCooldownGameplayEffect())   // 共用的 GE_Cooldown（时
    {
        FGameplayEffectSpecHandle Spec = MakeOutgoingGameplayEffectSpec(CooldownGE->GetClass()
        Spec.Data->DynamicGrantedTags.AppendTags(CooldownTags);       // 动态授予本技能的冷却 Ta
        Spec.Data->SetSetByCallerMagnitude(MyTags::SetByCaller_Cooldown, CooldownDuration.GetV
        ApplyGameplayEffectSpecToOwner(Handle, ActorInfo, ActivationInfo, Spec);   // 会带上预
    }
}
```

> 🕐 **版本差异**
>
> `FGameplayEffectSpec::DynamicGrantedTags` 在 5.3 之后仍然可用，但引擎更推荐通过 `Spec.AddDynamicGrantedTag()` 之类的接口访问（不同小版本的接口名有差异，以头文件为准）。思路不变：**同一个 GE，在运行时授予不同的 Tag。**

8.4 四个策略：决定技能在网络中的行为

策略	选项	建议
InstancingPolicy 实例化	`InstancedPerActor`：每个 ASC 一个实例，可以保存状态 `InstancedPerExecution`：每次激活新建实例 `NonInstanced`：使用 CDO（**5.5 废弃**）	默认使用 **InstancedPerActor**。PerExecution 每次激活都要分配对象，开销较大，只在确实需要多个实例同时运行时使用
NetExecutionPolicy 在哪里执行	`LocalPredicted`：客户端先执行，服务器确认 `LocalOnly`：只在本地客户端执行 `ServerInitiated`：服务器先执行，再通知客户端 `ServerOnly`：只在服务器执行	玩家主动技能用 **LocalPredicted**；被动、AI 技能、死亡技能用 ServerInitiated / ServerOnly；纯本地 UI 行为（例如打开瞄准镜）用 LocalOnly
NetSecurityPolicy 谁能发起 / 终止	`ClientOrServer` `ServerOnlyExecution`：客户端不能请求激活 `ServerOnlyTermination`：客户端不能取消 `ServerOnly`：两者都由服务器决定	"复活""使用道具结算"这类敏感技能，用 ServerOnlyExecution 防止客户端伪造激活请求
ReplicationPolicy 实例本身是否复制	`ReplicateNo` / `ReplicateYes`	保持 **ReplicateNo**。激活与结束本来就通过 RPC 同步，与这个选项无关；5.5 起 GA 上的 Replicated 属性已废弃，跨端传值请使用 RPC

> 🌐 **网络视角：技能实例只存在于两个地方**
>
> 即使设置了 ReplicateYes，GA 实例也只存在于**服务器**和**拥有者客户端**。其他客户端（模拟端）上没有技能实例，它们看到的动画来自 Montage 复制，特效来自 Cue，状态来自 Tag。这一点决定了第三部分的很多设计。

8.5 激活方式

方式	API / 配置	适用场景
句柄	`TryActivateAbility(SpecHandle)`	输入绑定（推荐配合 InputTag）
Tag	`TryActivateAbilitiesByTag(Container)`，匹配 AssetTags	"跳跃""闪避"这类通用动作
类	`TryActivateAbilityByClass`	不推荐，会和具体类耦合
GameplayEvent	GA 的 `AbilityTriggers` 配置 TriggerSource = GameplayEvent；`SendGameplayEventToActor` 触发	受击反应、死亡、连招衔接；事件 Payload 会传给 `ActivateAbilityFromEvent`
Tag 增删	TriggerSource = OwnedTagAdded / OwnedTagPresent	"获得 State.Debuff.Stun 时自动播放眩晕技能"，Tag 消失时 Present 类型的技能会被取消
被动	在 `OnAvatarSet` 或 `OnGiveAbility` 中调用 `TryActivateAbility`	常驻被动（Lyra 中的 ActivationPolicy = OnSpawn）

8.6 一个完整的 C++ 技能：火球术

GA_Fireball.cpp

```cpp
UGA_Fireball::UGA_Fireball()
{
    InstancingPolicy   = EGameplayAbilityInstancingPolicy::InstancedPerActor;
    NetExecutionPolicy = EGameplayAbilityNetExecutionPolicy::LocalPredicted;
    // AssetTags / ActivationBlockedTags 等一般在蓝图子类中配置
}

void UGA_Fireball::ActivateAbility(const FGameplayAbilitySpecHandle Handle, const FGameplayAbi
                                   const FGameplayAbilityActivationInfo ActivationInfo, const
{
    // ① 提交：扣蓝 + 进冷却（在预测窗口内，客户端会预测执行）
    if (!CommitAbility(Handle, ActorInfo, ActivationInfo))
    {
        EndAbility(Handle, ActorInfo, ActivationInfo, true, true);
        return;
    }

    // ② 播放施法动画（本地立即播放；服务器播放后通过 Montage 复制给其他客户端）
    UAbilityTask_PlayMontageAndWait* MontageTask =
        UAbilityTask_PlayMontageAndWait::CreatePlayMontageAndWaitProxy(this, NAME_None, CastMo
    MontageTask->OnCompleted.AddDynamic(this, &ThisClass::OnMontageFinished);
    MontageTask->OnBlendOut.AddDynamic(this, &ThisClass::OnMontageFinished);
    MontageTask->OnInterrupted.AddDynamic(this, &ThisClass::OnMontageCancelled);
    MontageTask->OnCancelled.AddDynamic(this, &ThisClass::OnMontageCancelled);
    MontageTask->ReadyForActivation();

    // ③ 等动画中的出手帧通知（AnimNotify 中调用 SendGameplayEventToActor(Event.Montage.Fire)）
    UAbilityTask_WaitGameplayEvent* FireTask =
        UAbilityTask_WaitGameplayEvent::WaitGameplayEvent(this, MyTags::Event_Montage_Fire, nu
    FireTask->EventReceived.AddDynamic(this, &ThisClass::OnFireEvent);
    FireTask->ReadyForActivation();
}

void UGA_Fireball::OnFireEvent(FGameplayEventData Payload)
{
    // ④ 只在服务器生成投射物（投射物是复制的 Actor）
    if (!HasAuthority(&CurrentActivationInfo)) return;

    const FTransform SpawnTM = GetMuzzleTransform();
    AMyProjectile* P = GetWorld()->SpawnActorDeferred<AMyProjectile>(ProjectileClass, SpawnTM,
                                                                     Cast<APawn>(GetAvatarActo
    // 把要施加的伤害 Spec 交给投射物，命中时直接 Apply
    FGameplayEffectSpecHandle Spec = MakeOutgoingGameplayEffectSpec(DamageEffect, GetAbilityLe
```

```
    Spec.Data→SetSetByCallerMagnitude(MyTags::SetByCaller_Damage, BaseDamage.GetValueAtLevel(
    P→DamageSpec = Spec;
    P→FinishSpawning(SpawnTM);
}

void UGA_Fireball::OnMontageFinished()  { EndAbility(CurrentSpecHandle, CurrentActorInfo, Curr
void UGA_Fireball::OnMontageCancelled() { EndAbility(CurrentSpecHandle, CurrentActorInfo, Curr
```

🔬 深入源码：CommitAbility 做了什么

`CommitAbility` = `CommitCheck`（再次检查冷却和消耗，因为激活之后状态可能已经变化）+ `CommitExecute`（调用 `ApplyCooldown` 和 `ApplyCost`）。你也可以分开调用 `CommitAbilityCost` 和 `CommitAbilityCooldown`，例如"引导技能开始时扣蓝，结束后才进入冷却"。`EndAbility` 和 `CancelAbility` 都**不会**自动提交。

本章小结

- 生命周期：Give → TryActivate → CanActivate → PreActivate → **ActivateAbility** → Commit → Tasks → **End / Cancel**。每个出口都必须调用 EndAbility。
- 十个 Tag 字段描述技能之间以及技能与角色状态之间的互斥关系，眩晕、沉默等控制效果都通过它们实现。
- 消耗和冷却都是 GE；冷却通过 Tag 判断，常见做法是所有技能共用一个冷却 GE，在运行时注入时长和 Tag。
- 默认使用 **InstancedPerActor + LocalPredicted + ReplicateNo**。

CHAPTER 09

AbilityTask 与 TargetData：技能中的异步

技能几乎都要"等待"：等动画、等出手帧、等玩家松手、等选择目标。AbilityTask 就是 GAS 用来处理这些等待的工具。

9.1 Task 是技能里的 await

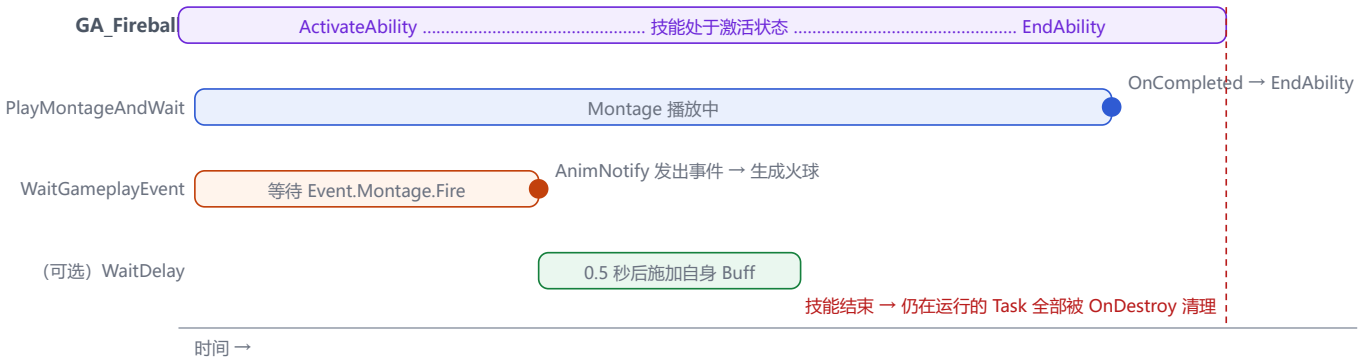


图 9-1 一个技能同时运行多个 Task，每个 Task 通过委托通知技能；技能结束时所有 Task 一起被销毁

与普通的计时器或委托相比，Task 有三点好处：①**生命周期与技能绑定**，技能结束时自动清理；②**与预测系统集成**，部分 Task 会在回调时开启新的预测窗口；③在蓝图中显示为带多个输出引脚的异步节点，策划也能看懂。

9.2 常用的内置 Task

Task	作用	网络说明
`PlayMontageAndWait`	播放 Montage，等待完成 / 中断 / BlendOut	通过 ASC 的 RepAnimMontageInfo 复制给模拟端
`WaitGameplayEvent`	等待发给自己的 GameplayEvent	两端各自等待；事件一般来自两端都会执行的 AnimNotify
`WaitTargetData`	生成 TargetActor 让玩家选择目标	客户端产生的 TargetData 通过 RPC 发给服务器
`WaitInputPress` / `WaitInputRelease`	等待再次按下或松开激活键	客户端回调时会开启新的预测窗口，并把输入事件同步给服务器
`WaitGameplayTagAdded` / `Removed`	等待自身 Tag 变化	两端独立判断
`WaitAttributeChange`	等待属性变化	客户端以复制值为准
`WaitDelay`	延时	两端各自计时
`ApplyRootMotionConstantForce` 等	Root Motion 位移（冲刺、击退）	接入 CharacterMovement 的移动预测，详见 13.3
`WaitNetSync`	两端互相等待对方执行到同一位置	会增加一个往返的延迟，按需使用
`WaitConfirmCancel`	等待确认或取消输入	配合 TargetData 使用

9.3 自定义 Task：蓄力

AbilityTask_Charge.h / .cpp

```cpp
DECLARE_DYNAMIC_MULTICAST_DELEGATE_OneParam(FChargeDelegate, float, ChargeRatio);

UCLASS()
class UAbilityTask_Charge : public UAbilityTask
{
    GENERATED_BODY()
public:
    UPROPERTY(BlueprintAssignable) FChargeDelegate OnProgress;   // 每帧回调：蓄力进度 0~1
    UPROPERTY(BlueprintAssignable) FChargeDelegate OnFullyCharged;

    // 蓝图节点工厂函数：DefaultToSelf/HidePin 让蓝图里不显示 OwningAbility 引脚
    UFUNCTION(BlueprintCallable, Category = "Ability|Tasks", meta = (HidePin = "OwningAbility"
    static UAbilityTask_Charge* WaitCharge(UGameplayAbility* OwningAbility, float MaxChargeTim

    virtual void Activate() override;
    virtual void TickTask(float DeltaTime) override;

private:
    float MaxTime = 1.f;
    float Elapsed = 0.f;
};

UAbilityTask_Charge* UAbilityTask_Charge::WaitCharge(UGameplayAbility* OwningAbility, float Ma
{
    UAbilityTask_Charge* Task = NewAbilityTask<UAbilityTask_Charge>(OwningAbility);
    Task->MaxTime = FMath::Max(0.01f, MaxChargeTime);
    Task->bTickingTask = true;                // 需要每帧 Tick
    return Task;
}

void UAbilityTask_Charge::Activate() { Elapsed = 0.f; }

void UAbilityTask_Charge::TickTask(float DeltaTime)
{
    Super::TickTask(DeltaTime);
    Elapsed += DeltaTime;
    const float Ratio = FMath::Min(Elapsed / MaxTime, 1.f);
    // 技能可能已经结束：这里统一检查后再广播
    if (ShouldBroadcastAbilityTaskDelegates()) { OnProgress.Broadcast(Ratio); }
    if (Ratio ≥ 1.f)
    {
        if (ShouldBroadcastAbilityTaskDelegates()) { OnFullyCharged.Broadcast(1.f); }
        EndTask();
```

```
    }
}
```

⚠️ **坑点**

- 在 C++ 中创建 Task 后**必须调用** `ReadyForActivation()`，否则 `Activate` 不会执行（蓝图节点会自动调用）。
- 广播委托之前检查 `ShouldBroadcastAbilityTaskDelegates()`，避免技能结束后回调还在执行。
- "按住蓄力、松手释放"应该组合使用 `WaitInputRelease` 和这个 Task。松手的时机由 `WaitInputRelease` 同步给服务器，不要在 Task 里自己读取输入。

9.4 TargetData：技能的"选择目标"

`FGameplayAbilityTargetDataHandle` 是一个可复制的多态容器，里面可以装命中结果（`FGameplayAbilityTargetData_SingleTargetHit`）、Actor 列表（`_ActorArray`）、位置信息（`_LocationInfo`），也可以装自定义结构体。它主要解决一个问题：**目标只有客户端最清楚**（玩家的准星、点击的位置），但结算必须在服务器进行。

ConfirmationType	含义
Instant	立即生成 TargetData（射线检测、自身周围范围）
UserConfirmed	等待玩家确认（地面指示器 → 点击释放）
Custom / CustomMulti	由 TargetActor 自己决定何时产出（例如可以多次产出的锁定类技能）

✅ **实践建议**

内置的 `AGameplayAbilityTargetActor` 体系比较笨重（每次都要生成 Actor）。射击类游戏可以参考 Lyra 的 `LyraGameplayAbility_RangedWeapon`：客户端自己做射线检测，打包成 TargetData，通过 `ServerSetReplicatedTargetData` 发给服务器，服务器校验后再结算。完整流程见 13.4。

本章小结

- Task 是技能里的异步步骤，生命周期与技能绑定，并且与预测系统集成。
- 自定义 Task 的要点：工厂函数用 `NewAbilityTask` 创建，需要 Tick 时设置 `bTickingTask`，广播前检查，结束时调用 `EndTask`。
- TargetData 把"客户端选择的目标"安全地交给服务器结算。

CHAPTER 10

GameplayCue：逻辑与表现分离

特效、音效、震屏、受击闪白都属于表现。Cue 让表现资源和逻辑代码完全解耦，并提供了一套省带宽、可预测的同步方式。

10.1 工作原理

Cue 通过 Tag 路由：所有 Cue Tag 必须以 `GameplayCue.` 开头。`UGameplayCueManager` 在启动时扫描配置的目录，建立"Tag → Notify 类"的映射表。逻辑层只需要说"触发 `GameplayCue.Hero.Q.Hit`"，不需要知道具体播放什么粒子或音效。

DefaultGame.ini

```
[/Script/GameplayAbilities.AbilitySystemGlobals]
; 只扫描这些目录，能明显缩短启动时间
+GameplayCueNotifyPaths="/Game/GAS/Cues"
```

10.2 四种事件



图 10-1 Cue 的事件类型。OnActive 只在"亲眼看到施加"的客户端触发，WhileActive 对中途进入视野的客户端也会触发

✅ 实践建议

循环特效放在 **WhileActive** 中启动，而不是 OnActive。否则后加入游戏或者刚进入视野的玩家，看不到已经在燃烧的目标身上的火焰。OnActive 只用于"施加瞬间"的一次性表现。

10.3 Notify 的类型

类型	实例化	适合
`UGameplayCueNotify_Static`	不实例化，在 CDO 上执行	一次性 Executed：命中火花、跳字、音效
`UGameplayCueNotify_Burst`	同上，可在编辑器里直接配置粒子、音效、震屏	一次性表现，不需要写代码
`AGameplayCueNotify_Actor`	在目标上生成 Actor，可以保存状态	持续表现：燃烧、护盾、光环。`bAutoDestroyOnRemove`，支持对象池回收
`AGameplayCueNotify_Looping`	Actor 类型，内置开始 / 循环 / 结束三段配置	标准的循环表现
`AGameplayCueNotify_BurstLatent`	Actor 类型	需要持续一段时间的一次性表现（带延时）

10.4 触发方式

方式	同步方式	可预测	说明
写在 GE 的 GameplayCues 列表中	随 GE 生命周期：Instant 走多播，持续型随 GE 复制	**是**	**首选。**参数（Magnitude、命中点）自动从 Spec 中获取
GA 中调用 `K2_ExecuteGameplayCue` / `K2_AddGameplayCue`	服务器多播；本地预测时客户端会先执行	**是**	不需要 GE 的纯表现，例如挥刀拖尾
ASC 的 `ExecuteGameplayCue` / `AddGameplayCue`	服务器 → Unreliable 多播	**视调用位置而定**	技能之外的通用入口
`UGameplayCueManager::ExecuteGameplayCue_NonReplicated` 等	**不同步**，只在本地执行	—	每个客户端都能自己判断的表现（例如本地计算出的命中特效）

自定义 Static Cue：命中跳字

```
UCLASS()
class UGCN_DamageNumber : public UGameplayCueNotify_Static
{
    GENERATED_BODY()
public:
    UGCN_DamageNumber() { GameplayCueTag = FGameplayTag::RequestGameplayTag("GameplayCue.UI.Da

    virtual bool OnExecute_Implementation(AActor* Target, const FGameplayCueParameters& Params
    {
        // RawMagnitude：GE 中第一个 Modifier 的数值（或手动传入）；EffectContext 中有命中点、来源
        const float Damage = Params.RawMagnitude;
        const FVector Loc = Params.EffectContext.GetHitResult() ? FVector(Params.EffectContext
                                                                : Target->GetActorLocation();
        if (UMyDamageNumberSubsystem* Sys = UWorld::GetSubsystem<UMyDamageNumberSubsystem>(Tar
        {
            Sys->Spawn(Loc, Damage);
        }
        return true;
    }
};
```

10.5 网络视角：Cue 是怎么到达每个客户端的

🌐 三条同步路径

- **Executed（一次性）**：服务器调用 ASC 上的 `NetMulticast_InvokeGameplayCueExecuted*` 系列 **Unreliable 多播**。网络拥塞时可能丢失，这对表现来说是可以接受的。
- **持续型（Add / Remove）**：拥有者通过复制下来的 ActiveGE 得知；模拟端通过 ASC 上的 `MinimalReplicationGameplayCues`（Mixed / Minimal 模式）得知，后者是可靠的属性复制，保证最终一致。
- **预测去重**：本地预测的客户端已经提前播放了 Cue。服务器多播时会带上预测键，拥有者识别出这是自己预测过的 Cue，就会**跳过**，避免重复播放。

⚠️ 坑点：同一帧大量 Cue 被丢弃

AOE 一次命中 20 个目标时，每个目标都会发出一个 Executed 多播，可能被网络层的 RPC 限流丢掉。可以考虑：①合并成一个 Cue，在参数中携带多个目标；②改为本地非复制 Cue，由每个客户端根据复制下来的结果自己判断；③检查项目中与多播 RPC 数量相关的网络配置。

✅ 实践建议：皮肤特效替换

同一个技能在不同皮肤下要播放不同特效时，逻辑层的 Cue Tag 应该保持不变，由表现层做映射。GAS 提供了 `GameplayCueTranslator` 机制，可以按条件把 `GameplayCue.Hero.Q.Hit` 翻译成 `GameplayCue.Hero.Q.Hit.Skin01`；也可以在 Notify 内部根据 Instigator 的皮肤配置选择资源。这样逻辑和数值完全不需要关心皮肤。

本章小结

- Cue 由 Tag 驱动，逻辑层只负责触发 Tag，具体表现由 Notify 实现。
- 事件分为 OnActive / WhileActive / Removed / Executed；循环表现放在 WhileActive 中。
- 一次性 Cue 走 Unreliable 多播，持续型 Cue 走可靠复制；预测过的 Cue 会自动去重。
- 首选在 GE 上配置 Cue，以获得自动参数和预测支持。

PART THREE

第三部分　网络同步：把联网讲透

单机写得对的技能，联网后往往问题不断。这一部分先讲"谁会收到什么数据"，再讲"预测键如何让客户端先执行、后对账"，最后用十个专题覆盖实际项目中最常见的网络问题。

CHAPTER 11

复制基础：谁会收到什么

GAS 的每一类数据都有自己的复制方式和接收范围。很多"模拟端不对"的问题，原因只是那份数据根本没有发给模拟端。

11.1 三种网络角色

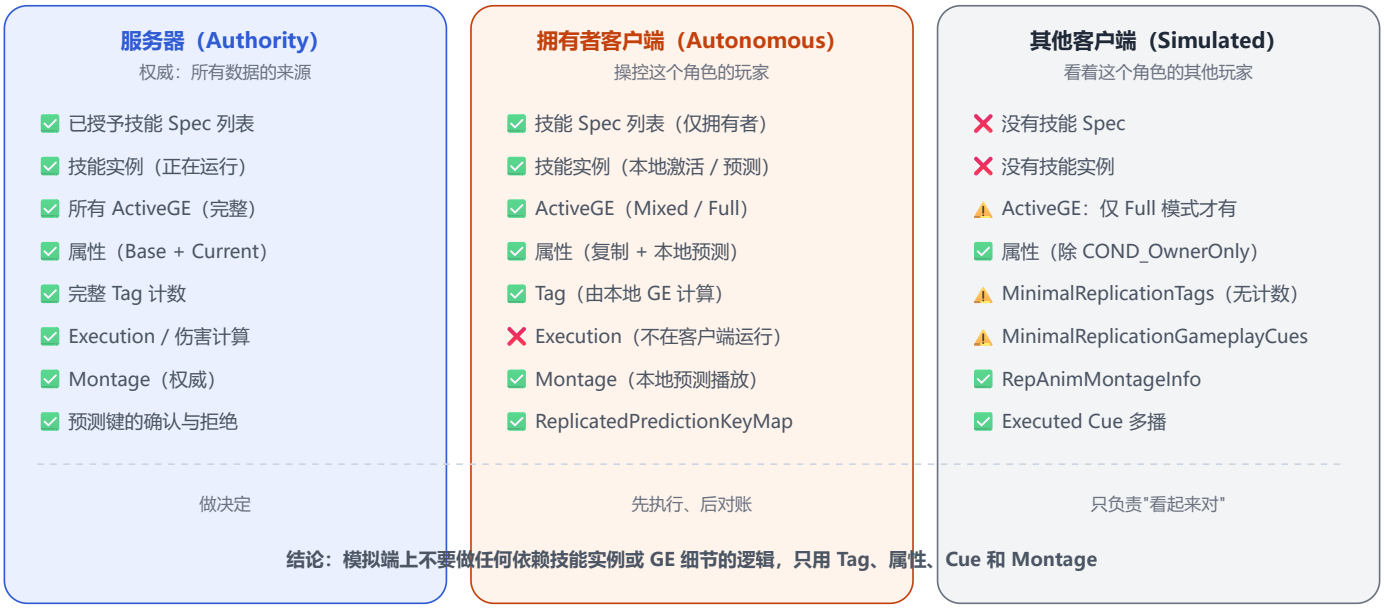


图 11-1 以 Mixed 模式为例，三种网络角色分别拥有哪些 GAS 数据

11.2 复制模式矩阵

数据	复制机制	Full	Mixed	Minimal
ActiveGameplayEffects	FastArray 属性复制	所有人	仅拥有者	**不复制**
MinimalReplicationTags	属性复制（只同步有无）	—	模拟端	所有人
MinimalReplicationGameplayCues	FastArray 属性复制	—	模拟端	所有人
Attributes	AttributeSet 子对象的属性复制	与模式无关，由每个属性的 DOREPLIFETIME 条件决定		
ActivatableAbilities（Spec 列表）	FastArray，条件为拥有者	始终只发给拥有者		
RepAnimMontageInfo	属性复制	发给模拟端		
技能激活 / 结束 / TargetData	Reliable RPC（Server ↔ 拥有者）	与模式无关		
Executed Cue	Unreliable 多播	与模式无关		

✅ **选择建议**

- **玩家：Mixed**（默认选择）。自己的 Buff 列表、剩余时间在 UI 上要显示完整信息，别人只需要知道状态和特效。
- **AI 小兵 / 野怪：Minimal**。没有任何客户端需要它们的 GE 细节。
- **Full**：只有在必须显示"其他玩家 Buff 的剩余时间"时才使用（例如 MOBA 中点击敌方英雄查看 Buff 倒计时），并且要做带宽分析。

11.3 FastArray：增量复制

ActiveGE 列表和技能 Spec 列表都使用 `FFastArraySerializer`，只复制**新增、修改、删除**的元素，而不是每次复制整个数组。客户端通过元素上的回调感知变化：

FastArray 回调	ActiveGE 在客户端上做的事
`PostReplicatedAdd`	把 Modifier 加入本地聚合器、添加 Granted Tag、触发 OnActive / WhileActive Cue（如果这个 GE 是自己预测过的，则跳过 OnActive）
`PostReplicatedChange`	层数、持续时间变化：重新计算、广播层数变化委托
`PreReplicatedRemove`	移除 Modifier 和 Tag、触发 Removed Cue

> 🔬 **深入源码：客户端的 CurrentValue 是怎么算出来的**
>
> AttributeSet 复制下来的是整个 `FGameplayAttributeData`（Base 和 Current 都在里面）。`GAMEPLAYATTRIBUTE_REPNOTIFY` 宏会调用 `SetBaseAttributeValueFromReplication`：如果客户端上存在这个属性的聚合器（也就是本地知道有哪些 Modifier，包括预测中的），就**用服务器的 Base 加上本地已知的 Modifier 重新计算 Current**；否则直接使用复制下来的值。这就是拥有者客户端能"预测属性"的基础。

11.4 时间：持续时间与冷却

ActiveGE 复制时带有服务器上的开始时间。客户端计算剩余时间时，用的是 **GameState 同步的服务器时间**（`GetServerWorldTimeSeconds()`），因此不同客户端看到的倒计时大致一致。`GetActiveEffectsTimeRemainingAndDuration` 已经处理好了这一点，UI 直接使用即可，不要自己用本地时间去推算。

11.5 RPC 的前提：NetOwner

ASC 上所有的 Server RPC（`ServerTryActivateAbility`、`ServerSetReplicatedTargetData` 等）只有在客户端**拥有**这个 ASC 所在的 Actor 时才能发出。所以：

- 玩家去攻击一个宝箱时，**不能**直接激活宝箱 ASC 上的技能（客户端不拥有宝箱）。应该在**玩家自己的** ASC 上激活"开宝箱"技能，由这个技能对宝箱施加效果。
- AI 的 ASC 没有拥有者客户端，所有技能都由服务器激活。

本章小结

- 服务器有全部数据；拥有者有自己的技能和 GE；模拟端只有属性、最小化 Tag、Cue 和 Montage。
- 玩家用 Mixed，小兵用 Minimal，Full 要谨慎。
- 客户端的 Current = 服务器 Base + 本地已知的 Modifier（包括预测中的），这是属性预测的基础。

CHAPTER 12

客户端预测：PredictionKey 全流程

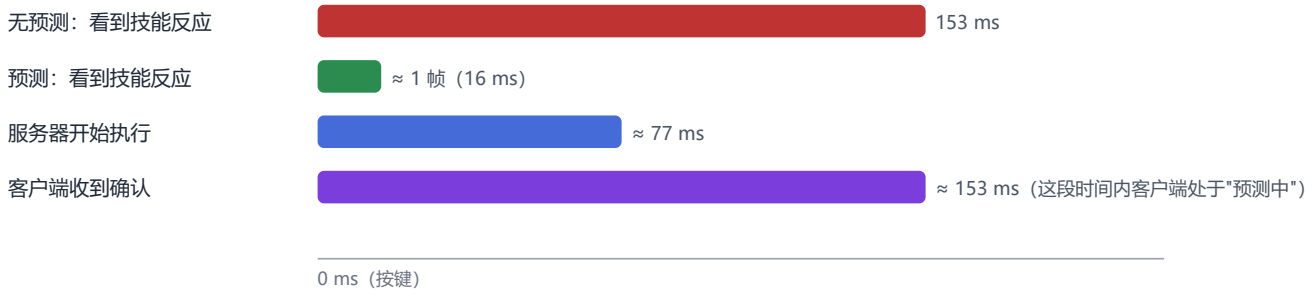
GAS 最有价值的部分。本章按时间顺序跟踪一次预测从生成、执行、确认到对账的过程，以及被拒绝时如何回滚。

12.1 为什么需要预测

没有预测时，玩家按下技能键，要等请求到达服务器、服务器执行完、结果回到客户端，才能看到反应，至少要经过一个 RTT（往返时间）。100ms 的延迟就足以让操作感觉"发黏"。预测的思路是：**客户端先按自己的判断执行，同时告诉服务器；服务器执行后告诉客户端结果是否正确，客户端再按服务器结果对账。**

交互实验：延迟对手感的影响

网络 RTT： **120** ms　服务器帧间隔： 33 ms



12.2 核心概念

FPredictionKey

一个由客户端生成的整数 ID。客户端在预测期间做的每一件事（施加 GE、加 Tag、播 Montage、触发 Cue）都会关联这个 Key。服务器执行同样的操

预测窗口（FScopedPredictionWindow）

一个 RAII 作用域。只有在窗口内，ASC 才有一个"当前有效的预测 Key"。技能激活时引擎会自动

作时使用**同一个** Key，客户端据此把"自己预测的"和"服务器发来的"对应起来。

打开窗口；部分 Task 回调（WaitInputRelease、WaitTargetData 等）会打开新的窗口。

确认（Confirm）与追平（CatchUp）

服务器处理完带 Key 的请求后，把 Key 写入 `ReplicatedPredictionKeyMap` 复制给客户端。客户端收到后触发 CaughtUp 委托：移除自己预测的副作用，以服务器复制下来的版本为准。

拒绝（Reject）

服务器判断激活失败时，发送 `ClientActivateAbilityFailed`，客户端触发 Rejected 委托，撤销该 Key 关联的所有预测副作用。

> 🌐 **Key 只对两端有意义**
>
> `FPredictionKey` 的 NetSerialize 只会把真实的值发给**生成它的那个客户端连接**，发给其他客户端的是无效 Key。所以模拟端完全不知道预测的存在，它们只会看到服务器的最终结果。

12.3 成功的预测：时序图

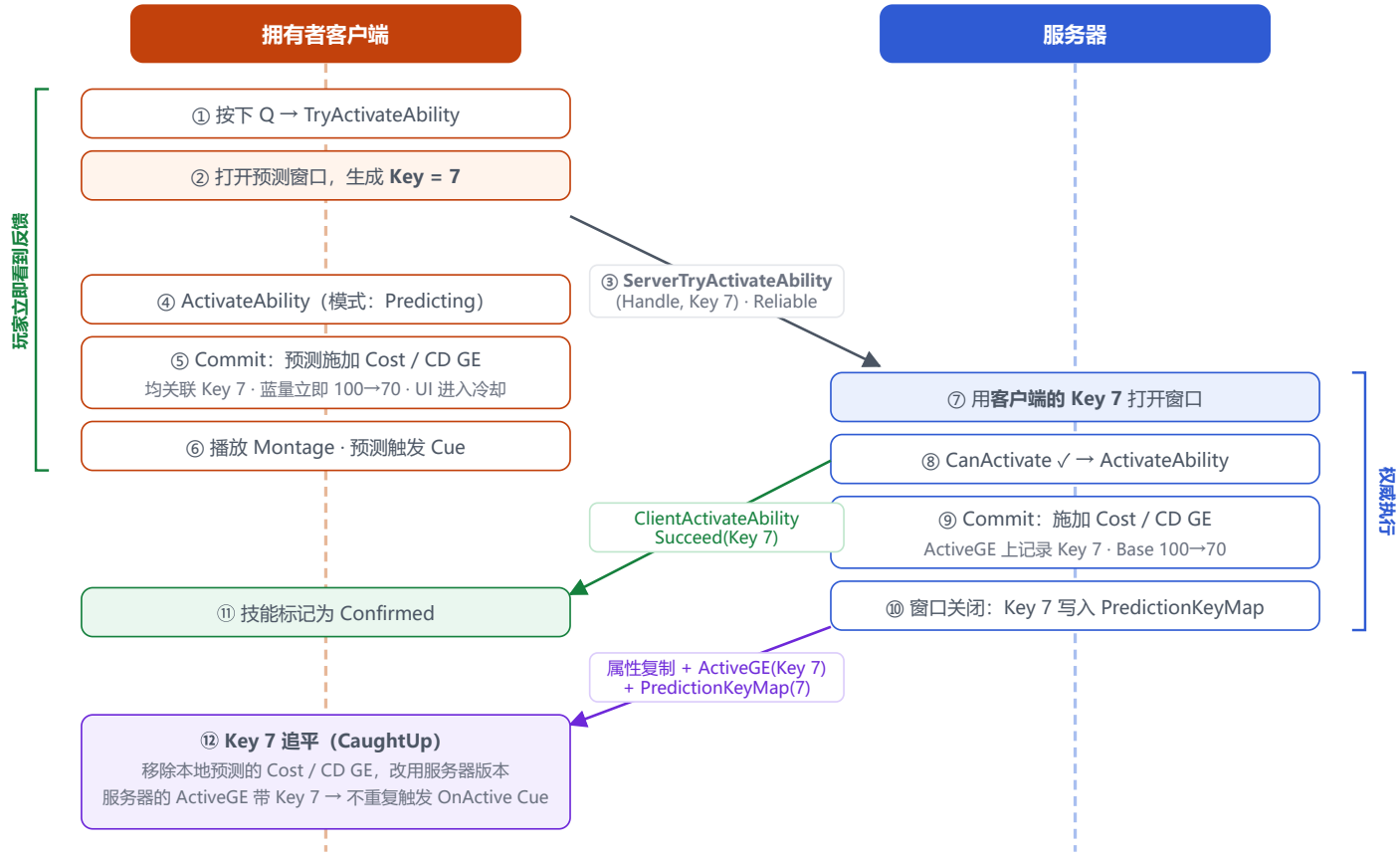


图 12-1 一次成功的本地预测：客户端第 ②–⑥ 步不需要等待网络，第 ⑫ 步用服务器的结果替换预测结果

🔬 **深入源码：关键函数**

客户端：`InternalTryActivateAbility` 中创建 `FScopedPredictionWindow(this, true)` 生成新 Key，先调用 `CallServerTryActivateAbility`，再调用 `CallActivateAbility`；服务器：`ServerTryActivateAbility_Implementation → InternalServerTryActivateAbility`，用收到的 Key 构造 `FScopedPredictionWindow(this, PredictionKey)`，窗口析构时把 Key 写入 `ReplicatedPredictionKeyMap`。`GameplayPrediction.h` 文件头部有 Epic 写的长篇设计说明，**强烈建议通读一遍。**

12.4 对账：蓝量为什么不会扣两次

这是理解预测最好的例子。Cost GE 是 Instant，会修改 Base；但客户端预测的修改必须是**可以撤销的**。GAS 的做法是：**客户端预测的 Instant GE 会被当作 Infinite GE 处理**，它不修改 Base，而是作为一个 Modifier 挂在聚合器上。

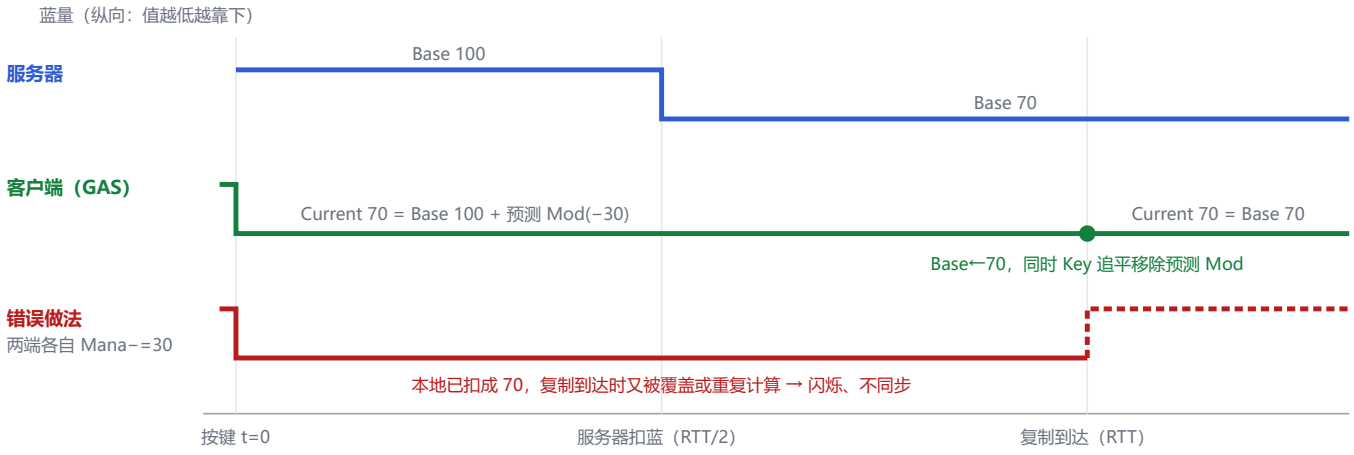


图 12-2 预测蓝量：客户端的预测 Modifier 挂在 Base 之上，确认后由服务器的 Base 替换，整个过程数值始终是 70

> ⚠️ **这就是 REPNOTIFY_Always 的原因**
>
> 由于有预测，客户端本地的属性值**可能已经等于**服务器复制下来的值。默认的复制规则是"值没变就不触发 OnRep"，这样 `GAMEPLAYATTRIBUTE_REPNOTIFY` 就不会执行，聚合器中的 Base 得不到更新。之后预测的 Modifier 被移除时，Current 会按旧的 Base 计算，结果就错了。`REPNOTIFY_Always` 保证每次复制都会把服务器的 Base 写入聚合器。

12.5 被拒绝的预测：回滚

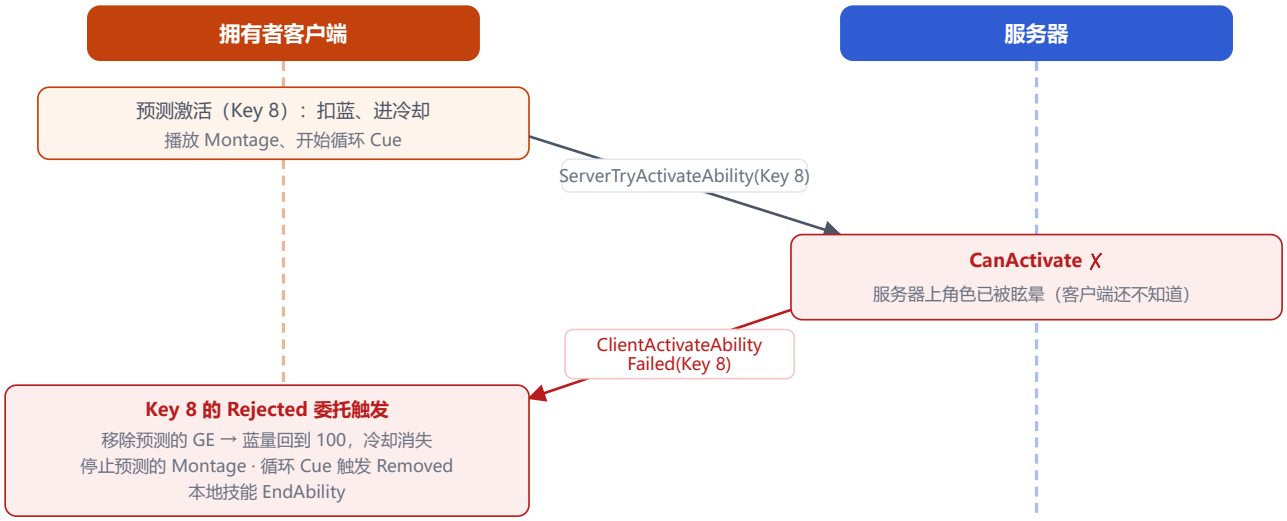


图 12-3 预测被拒绝：与 Key 8 关联的副作用被整体撤销

✅ **实践建议：让回滚不那么突兀**

回滚在玩家看来是"技能放了一半就消失了"。减少回滚的办法是**让客户端的判断尽量准确**：控制效果的 Tag 要尽快同步（调高 NetUpdateFrequency），冷却 UI 留一点容差，激活条件尽量依赖拥有者本地就能得到的数据。完全消除回滚是不可能的，这是预测的固有代价。

12.6 依赖 Key 与新的预测窗口

一个技能在运行过程中，可能需要**再次**预测：例如"按住蓄力、松手发射"，松手时要预测发射。这时 `WaitInputRelease` 在客户端的回调中打开一个新的预测窗口，生成一个**依赖于原始 Key 的新 Key**（`FPredictionKey` 中的 Base 字段指向原始 Key），并通过 `ServerSetReplicatedEvent` 把松手事件和新 Key 一起发给服务器。如果原始 Key 被拒绝，依赖它的 Key 也会一起被拒绝。

⚠️ **坑点：在预测窗口之外施加 GE**

在技能中通过 `SetTimer`、普通委托或者自己写的异步回调去施加 GE，这时已经**不在任何预测窗口内**。客户端调用 `ApplyGameplayEffectSpecToSelf` 时会因为没有有效 Key 而**被直接忽略**（`HasNetworkAuthorityToApplyGameplayEffect` 返回 false）；服务器正常施加。结果是"客户端没有预测，要等复制"。需要预测的操作，请放在 `ActivateAbility` 的同步代码中，或者放在会打开预测窗口的 Task 回调中。

12.7 能预测什么、不能预测什么

能预测 ✅	不能预测 ❌
• 技能激活（包括由事件触发的激活） • GE 施加：属性修改、Tag 授予 • Cost / Cooldown（在窗口内 Commit 时） • GameplayCue（随 GE 或单独触发） • Montage 播放 • 移动（由 CharacterMovement 自身的预测系统负责，包括 RootMotion Task）	• **GE 的移除** • **Periodic GE 的周期跳动**（DOT 每秒掉血） • **Execution 计算**（伤害公式只在服务器运行） • 在窗口外执行的任何操作 • 对其他 Actor 的伤害结算（应以服务器为准） • 投射物的生成（需要额外实现"假弹道"等方案）

12.8 在代码中区分执行端

技能中常用的执行端判断

```
// 是否在服务器上（权威）
if (HasAuthority(&CurrentActivationInfo)) { /* 生成投射物、施加伤害、随机数 */ }

// 是否是"正在预测"的客户端（服务器确认前）
if (IsPredictingClient()) { /* 只做本地表现 */ }

// 是否本地控制（Listen Server 主机玩家也返回 true）
if (IsLocallyControlled()) { /* 摄像机抖动、准星动画、本地 UI */ }

// 当前激活模式：Authority / NonAuthority / Predicting / Confirmed / Rejected
switch (GetCurrentActivationInfo().ActivationMode) { /* ... */ }
```

本章小结

- 客户端在预测窗口中生成 Key，所有副作用都关联到这个 Key；服务器用同一个 Key 执行。
- 成功时：确认 → Key 追平 → 用服务器复制的数据替换预测的数据。失败时：Rejected 委托撤销所有副作用。
- 预测的 Instant GE 在客户端被当作 Infinite 处理，所以可以撤销；这也是属性要使用 REPNOTIFY_Always 的原因。

- GE 移除、Periodic tick、Execution 都不能预测；在预测窗口外施加的 GE 在客户端会被忽略。

CHAPTER 13

网络专题：十个常见问题

每个专题都对应实际项目中经常出现的一类网络问题，给出原因和解决方法。

13.1 专题一：冷却预测与 UI 抖动

现象：冷却圈在技能释放约 0.1 秒后"跳"了一下；或者冷却刚转完就按键，技能放出来又被撤回。

原因：客户端比服务器早 RTT/2 开始冷却。Key 追平时，预测的冷却 GE 被移除，改用服务器的版本，而服务器版本的开始时间更晚，剩余时间就会变长一点。同理，客户端的冷却也会比服务器早结束，玩家这时按键，服务器上冷却还没结束，于是拒绝了这次激活。

对策：

- UI 统一用 `GetActiveEffectsTimeRemainingAndDuration` 读取剩余时间（它基于服务器时间），并对显示值做平滑处理，不要直接显示突变。
- 冷却结束前一小段时间内（例如 RTT/2）的按键先**缓存**，冷却结束后再激活，即"输入缓冲"。这也是动作游戏常用的手法。
- 对于时长很短、要求高频释放的技能（例如普攻），可以在服务器上留一点容差：服务器上冷却剩余时间小于某个阈值时，仍然允许激活。

13.2 专题二：Montage 是怎么同步的



图 13-1 Montage 的三端同步路径，以及一个只在专用服务器上出现的问题

✅ 实践建议

在技能中播放动画，请始终使用 `PlayMontageAndWait`（底层是 `ASC::PlayMontage`），**不要**直接调用 `AnimInstance::Montage_Play`。后者既不会同步给模拟端，被拒绝时也不会回滚。

13.3 专题三：位移技能与移动预测

现象：冲刺技能在高延迟下"拉回"，或者模拟端看到的冲刺轨迹不对。

原因：GAS 的预测**不包括**移动。角色移动由 `CharacterMovementComponent`（CMC）自己的预测系统负责（SavedMoves + 服务器校正）。在技能中直接调用 `SetActorLocation` 或 `LaunchCharacter`，CMC 不知道这次位移，服务器校正时就会把角色拉回去。

做法	是否可预测	说明
`ApplyRootMotionConstantForce` / `MoveToForce` 等 Task	**是**	创建 RootMotionSource，交给 CMC 预测和同步，推荐
带 RootMotion 的 Montage	**是**	CMC 原生支持 Montage RootMotion 的预测
修改 MoveSpeed 属性（加速 Buff）	**部分**	CMC 的 `GetMaxSpeed` 读取属性时，属性变化不在 SavedMove 中，高延迟下会出现轻微校正。要求严格时，应该扩展 CMC 的 SavedMove 标志位
`SetActorLocation` / `LaunchCharacter`	**否**	只在服务器调用时会表现为延迟和拉扯

13.4 专题四：TargetData 的同步与校验

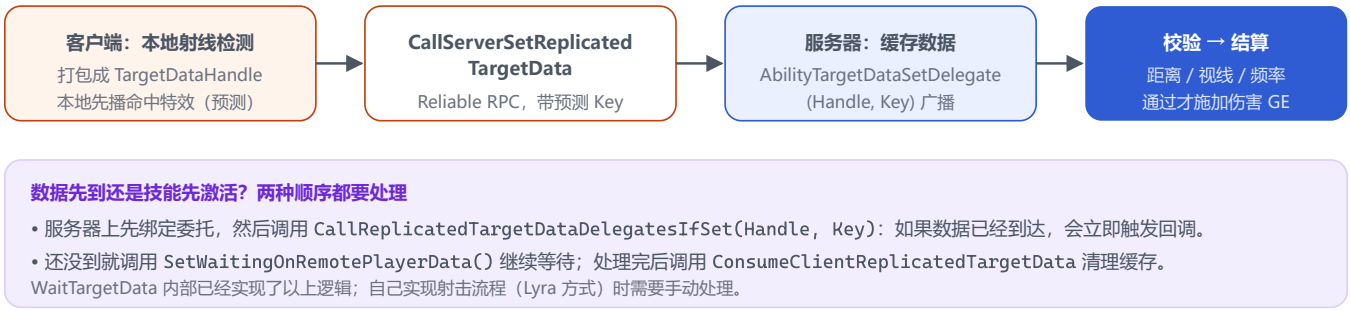


图 13-2 TargetData：客户端负责"瞄准"，服务器负责"校验和结算"

服务器端接收客户端 TargetData（Lyra 风格简化版）

```
void UGA_Fire::ActivateAbility( ... )
{
    UAbilitySystemComponent* ASC = CurrentActorInfo->AbilitySystemComponent.Get();
    if (HasAuthority(&CurrentActivationInfo) && !IsLocallyControlled())
    {
        // 服务器（且不是 Listen 主机自己）：等待客户端的 TargetData
        OnTargetDataReadyHandle = ASC->AbilityTargetDataSetDelegate(CurrentSpecHandle, Current
                                      .AddUObject(this, &ThisClass::OnTargetDataReady);
        ASC->CallReplicatedTargetDataDelegatesIfSet(CurrentSpecHandle, CurrentActivationInfo.G
    }
    else if (IsLocallyControlled())
    {
        FGameplayAbilityTargetDataHandle Data = TraceAndBuildTargetData();    // 本地射线检测
        FScopedPredictionWindow Window(ASC, IsPredictingClient());
        if (IsPredictingClient())
        {
            ASC->CallServerSetReplicatedTargetData(CurrentSpecHandle, CurrentActivationInfo.Ge
                                                   Data, FGameplayTag(), ASC->ScopedPredictior
        }
        OnTargetDataReady(Data, FGameplayTag());    // 本地也执行一次：客户端只做表现，服务器（List
    }
}

void UGA_Fire::OnTargetDataReady(const FGameplayAbilityTargetDataHandle& Data, FGameplayTag Ta
{
    UAbilitySystemComponent* ASC = CurrentActorInfo->AbilitySystemComponent.Get();
    if (HasAuthority(&CurrentActivationInfo))
    {
        if (ValidateHits(Data))                    // ★ 反作弊：不要盲目相信客户端
        {
            ApplyGameplayEffectToTarget(CurrentSpecHandle, CurrentActorInfo, CurrentActivatior
        }
        ASC->ConsumeClientReplicatedTargetData(CurrentSpecHandle, CurrentActivationInfo.GetAct
    }
    PlayLocalHitEffects(Data);                     // 本地表现（非复制 Cue）
}
```

13.5 专题五：RPC 批处理

一个"点一下就打完"的射击技能，正常要发三个 RPC：激活、TargetData、结束。连发武器每秒十几发时，开销很可观。GAS 提供了 **RPC 批处理**，把这些合并成一个 `ServerAbilityRPCBatch`：

开启批处理

```
// 1) 自定义 ASC 中开启
virtual bool ShouldDoServerAbilityRPCBatch() const override { return true; }

// 2) 在激活技能的地方开启批处理作用域
{
    FScopedServerAbilityRPCBatcher Batcher(ASC, AbilitySpecHandle);
    ASC->TryActivateAbility(AbilitySpecHandle);    // 技能内部同步完成：发送 TargetData + EndAbil
}   // 作用域结束时，三个 RPC 合并成一个发出
```

> ⚠️ **适用前提**
>
> 批处理只在技能**在同一帧内同步完成**（激活 → TargetData → 结束）时才有意义。需要等待动画或输入的技能，没法合并。

13.6 专题六：GameplayEvent 不会自动同步

现象：客户端发送了 `SendGameplayEventToActor`，服务器上的 `WaitGameplayEvent` 一直收不到。

原因：GameplayEvent 只是**本地函数调用**，本身不会走网络。唯一的例外是：事件**触发了**一个本地预测技能的激活，这时 Payload 会随 `ServerTryActivateAbilityWithEventData` 一起发给服务器。

对策：需要两端都收到的事件，应该让两端**各自产生**（例如来自两端都会播放的 Montage 中的 AnimNotify，注意 13.2 中提到的专用服务器问题）；或者使用 `ServerSetReplicatedEvent` / 自定义的 Reliable RPC 传递。

13.7 专题七：WaitNetSync

有时需要保证两端执行到同一位置后再继续，例如"客户端决定了某个值，服务器必须先收到这个值再继续执行"。`WaitNetSync` 有三种模式：

模式	行为
OnlyServerWait	客户端发出信号后立即继续（打开新的预测窗口）；服务器等待收到信号后再继续
OnlyClientWait	服务器发出信号后继续；客户端等待
BothWait	两端互相等待（会产生一个完整 RTT 的停顿）

官方建议：GA 中不要使用 Replicated 属性传值（5.5 已废弃），因为属性复制和激活 RPC 之间没有顺序保证。同一 Actor 上的 **Reliable RPC 之间是有序的**，用 RPC + WaitNetSync 才能得到可靠的时序。

13.8 专题八：命中特效的预测

伤害结算在服务器，但射击游戏的命中火花必须**立即**出现。常见做法是：客户端根据本地射线检测的结果，用**非复制的本地 Cue** 播放命中效果；服务器结算后，模拟端通过服务器的 Cue 看到同样的效果，拥有者则跳过（避免重复）。伤害数字等依赖结算结果的表现，等服务器结果到达后再显示。

13.9 专题九：安全与反作弊

服务器会重新检查

即使是 LocalPredicted 技能，服务器收到激活请求后也会重新执行 `CanActivateAbility`（冷却、消耗、Tag）。客户端改内存跳过冷却，服务器仍会拒绝。

敏感技能由服务器激活

使用 `NetSecurityPolicy = ServerOnlyExecution`，或者 `NetExecutionPolicy = ServerOnly / ServerInitiated`，客户端无法请求激活。

校验 TargetData

客户端发来的命中目标必须校验：距离、视线、射速、目标是否存活。数值计算全部放在服务器的 Execution 中。

事件 Payload 不可信

由客户端事件激活的技能，Payload 来自客户端。不要直接把 Payload 中的数值（例如 EventMagnitude）当作伤害使用。

13.10 专题十：带宽、相关性与测试

- **PlayerState 始终相关**：PlayerState 默认 `bAlwaysRelevant`，ASC 放在 PlayerState 上时，属性和 Cue 会同步给**所有**客户端，与距离无关。玩家很多的模式（例如大逃杀）需要评估这部分带宽：把只有自己需要的属性设为 `COND_OwnerOnly`，或者让部分逻辑走 Pawn 上的数据。
- **属性复制条件**：蓝量、能量、经验等只需要自己看到的属性使用 `COND_OwnerOnly`。
- **小兵使用 Minimal 模式**，并且尽量少挂常驻 GE；可以用 Tag 表示的状态就不要用 GE。
- **Iris**：UE5 新的复制系统 Iris 已经对 GAS 做了适配，新项目可以评估，但请以所用引擎版本的实际支持情况为准。
- **测试**：PIE 中选择 *Play As Client*，并开启 *Network Emulation*（Bad 预设），或者在控制台输入 `Net PktLag=150`、`Net PktLoss=5`。`p.NetShowCorrections 1` 可以显示移动校正。**所有技能都应该在 150ms 延迟、5% 丢包的环境下测试一遍。**

本章小结

- 冷却：客户端比服务器早开始、早结束，用服务器时间显示，配合输入缓冲。
- 动画用 PlayMontageAndWait；位移用 RootMotion Task；专用服务器要确保动画 Tick。
- TargetData：客户端瞄准，服务器校验后结算；GameplayEvent 本身不同步。
- 上线前，所有技能都要在高延迟、丢包的模拟环境下测试。

PART FOUR

第四部分　实战：从零做一个 MOBA 英雄

把前面所有知识用在一个完整的联网英雄上：属性、伤害管线、普攻连招、被动叠层、位移、控制、地面 AOE、冷却 UI，并逐个分析每个技能的网络行为。

CHAPTER 14

实战：英雄「炎刃」

一份策划需求，从数据建模到每个技能的实现，再到联网测试清单。

14.1 需求卡

技能	描述	关键机制
普攻	三段连击，第三段伤害 ×1.5	Montage Section、AnimNotify 事件、服务器扫掠检测
被动 • 炽焰	普攻命中叠 1 层炽焰（最多 3 层，5 秒）；满层后下次普攻额外造成 60 + 40% 法强的魔法伤害并清空层数	常驻技能、Stacking、GameplayEvent
Q • 冲锋斩	向前冲刺 600 距离，对路径上的敌人造成物理伤害；冲刺中免疫控制；耗 50 蓝，冷却 8 秒	LocalPredicted、RootMotion、免疫、互斥 Tag
W • 晕眩飞刃	掷出飞刃，命中第一个敌人造成魔法伤害并眩晕 1 秒；冷却 10 秒	服务器投射物、控制 GE、打断技能
R • 烈焰领域	选择地面位置，生成 4 秒的火焰领域，区域内敌人持续受到灼烧；冷却 60 秒	TargetData（地面选择）、周期伤害、领域 Actor

14.2 整体结构

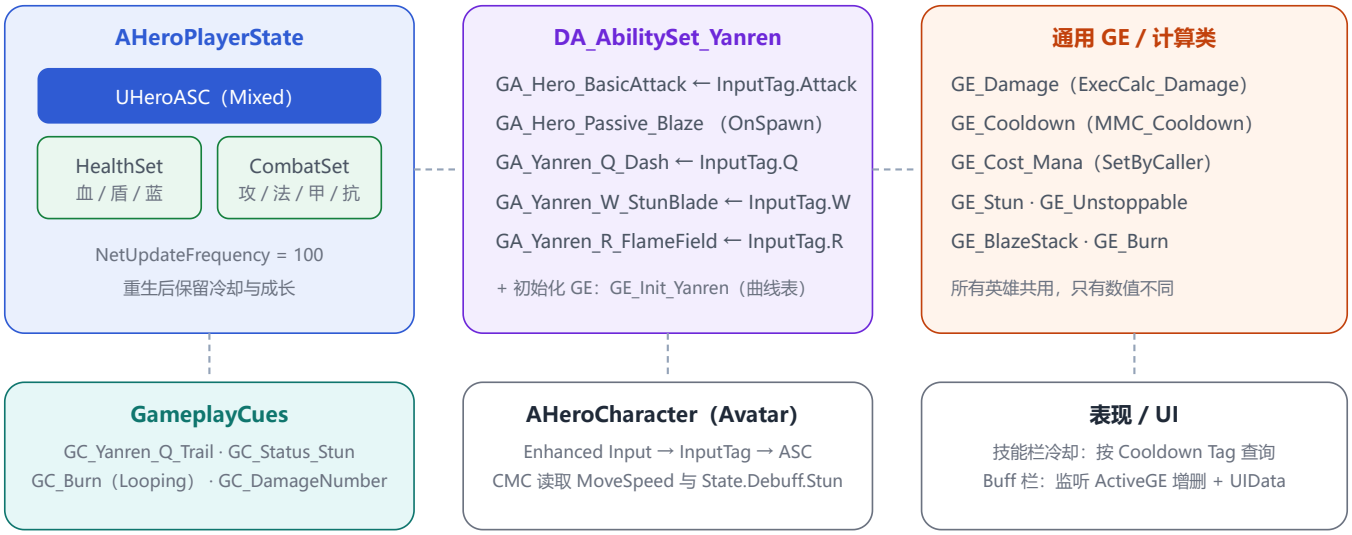


图 14-1 「炎刃」的模块划分：英雄特有的只有 AbilitySet 中的五个技能，GE、计算类和 Cue 大部分是全英雄共用的

14.3 数据建模

属性

属性集	属性	复制条件	说明
HealthSet	Health / MaxHealth	COND_None	所有人都要看血条
	Shield	COND_None	护盾值，伤害先扣护盾
	Mana / MaxMana	COND_OwnerOnly	只有自己需要看蓝量
CombatSet	AttackDamage / AbilityPower	COND_None	物攻 / 法强
	Armor / MagicResist / ArmorPen / MagicPen	COND_None	防御与穿透（百分比）
	CritChance / CooldownReduction / MoveSpeed	COND_None	CMC 读取 MoveSpeed
Meta	IncomingDamage / IncomingHeal	不复制	只在服务器使用的中转属性

Tag

类别	Tag
技能身份	`Ability.Type.Attack` `Ability.Type.Active` `Ability.Type.Passive` `Ability.Yanren.Q/W/R`
状态	`State.Dead` `State.Dashing` `State.Debuff.Stun` `State.Buff.Unstoppable`
冷却	`Cooldown.Yanren.Q/W/R`
GE 分类	`Effect.Type.CC` `Effect.Type.Damage` `Effect.RemoveOnDeath`
伤害类型	`Damage.Type.Physical` `Damage.Type.Magic`
事件	`Event.Montage.Hit` `Event.Attack.Landed` `Event.Death`
数值键	`SetByCaller.Damage` `SetByCaller.AttackCoef` `SetByCaller.APCoef` `SetByCaller.Cooldown`

14.4 伤害管线



数值示例：Q 冲锋斩（物理）

基础 `120 + 物攻 200 × 0.8 = 280`

目标护甲 `100`，穿透 `20%` → 有效护甲 `80` → 减伤乘数 `100/180 = 0.556` → `155.6`

目标护盾 `50` → 护盾 `50→0`，`Health −105.6`

全部发生在服务器；客户端只能通过属性复制和跳字 Cue 看到结果

图 14-2 所有伤害共用一条管线：技能只负责"给多少基础值、用什么类型"

ExecCalc_Damage —— 按伤害类型选择防御属性（在 7.3 的基础上扩展）

```
const FGameplayTagContainer& AssetTags = Spec.GetDynamicAssetTags();   // 技能通过 AddDynamicAs
const bool bMagic = AssetTags.HasTag(MyTags::Damage_Type_Magic);

float Defense = 0.f, Pen = 0.f, Power = 0.f;
ExecParams.AttemptCalculateCapturedAttributeMagnitude(bMagic ? S.MagicResistDef : S.ArmorDef,
ExecParams.AttemptCalculateCapturedAttributeMagnitude(bMagic ? S.MagicPenDef    : S.ArmorPenDe
ExecParams.AttemptCalculateCapturedAttributeMagnitude(bMagic ? S.AbilityPowerDef: S.AttackDama

const float Coef = Spec.GetSetByCallerMagnitude(bMagic ? MyTags::SetByCaller_APCoef : MyTags::
float Damage = Spec.GetSetByCallerMagnitude(MyTags::SetByCaller_Damage, false, 0.f) + Power *
Damage *= 100.f / (100.f + FMath::Max(0.f, Defense * (1.f - Pen)));
if (!bMagic && FMath::FRand() < CritChance) { Damage *= 2.f; }
```

HealthSet::PostGameplayEffectExecute —— 护盾优先

```
if (Data.EvaluatedData.Attribute == GetIncomingDamageAttribute())
{
    float Damage = GetIncomingDamage();
    SetIncomingDamage(0.f);
    if (Damage ≤ 0.f) return;

    const float Absorbed = FMath::Min(GetShield(), Damage);
    SetShield(GetShield() - Absorbed);
    Damage -= Absorbed;

    SetHealth(FMath::Clamp(GetHealth() - Damage, 0.f, GetMaxHealth()));

    // 跳字：在目标身上执行一个 Cue，RawMagnitude 填实际伤害
    FGameplayCueParameters CueParams;
    CueParams.RawMagnitude  = Damage + Absorbed;
    CueParams.EffectContext = Data.EffectSpec.GetContext();
    Data.Target.ExecuteGameplayCue(MyTags::GameplayCue_UI_DamageNumber, CueParams);

    if (GetHealth() ≤ 0.f && !Data.Target.HasMatchingGameplayTag(MyTags::State_Dead))
    {
        FGameplayEventData Ev; Ev.Instigator = Data.EffectSpec.GetContext().GetOriginalInstiga
        Data.Target.HandleGameplayEvent(MyTags::Event_Death, &Ev);   // 触发 GA_Death（ServerI
    }
}
```

14.5 普攻连招与被动「炽焰」

普攻是一个技能、三段 Section。玩家在连招窗口内再次按键时，跳转到下一段。命中检测只在服务器进行，命中后给**自己**发一个 `Event.Attack.Landed` 事件，被动技能监听这个事件，从而实现与普攻的解耦：

GA_Hero_Passive_Blaze.cpp

```
UGA_Hero_Passive_Blaze::UGA_Hero_Passive_Blaze()
{
    InstancingPolicy   = EGameplayAbilityInstancingPolicy::InstancedPerActor;
    NetExecutionPolicy = EGameplayAbilityNetExecutionPolicy::ServerOnly;    // 被动只在服务器运行
}

void UGA_Hero_Passive_Blaze::OnAvatarSet(const FGameplayAbilityActorInfo* ActorInfo, const FGa
{
    Super::OnAvatarSet(ActorInfo, Spec);
    // 获得技能 / 更换 Avatar 时自动激活（Lyra 中对应 ActivationPolicy = OnSpawn）
    if (ActorInfo && ActorInfo->IsNetAuthority() && !Spec.IsActive())
    {
        ActorInfo->AbilitySystemComponent->TryActivateAbility(Spec.Handle);
    }
}

void UGA_Hero_Passive_Blaze::ActivateAbility( ... )
{
    // 常驻：不 Commit，不 End；一直等待普攻命中事件
    UAbilityTask_WaitGameplayEvent* Wait = UAbilityTask_WaitGameplayEvent::WaitGameplayEvent(
        this, MyTags::Event_Attack_Landed, nullptr, /*OnlyTriggerOnce*/ false);
    Wait->EventReceived.AddDynamic(this, &ThisClass::OnAttackLanded);
    Wait->ReadyForActivation();
}

void UGA_Hero_Passive_Blaze::OnAttackLanded(FGameplayEventData Payload)
{
    UAbilitySystemComponent* ASC = GetAbilitySystemComponentFromActorInfo();
    const FGameplayEffectQuery Q = FGameplayEffectQuery::MakeQuery_MatchAnyOwningTags(FGamepla
    const int32 Stacks = ASC->GetAggregatedStackCount(Q);

    if (Stacks ≥ 3)
    {
        // 满层：对本次命中目标追加魔法伤害，并清空层数
        FGameplayEffectSpecHandle Spec = MakeOutgoingGameplayEffectSpec(DamageGE, GetAbilityLe
        Spec.Data->SetSetByCallerMagnitude(MyTags::SetByCaller_Damage, 60.f);
        Spec.Data->SetSetByCallerMagnitude(MyTags::SetByCaller_APCoef, 0.4f);
        Spec.Data->AddDynamicAssetTag(MyTags::Damage_Type_Magic);
        if (UAbilitySystemComponent* TargetASC = UAbilitySystemGlobals::GetAbilitySystemCompor
        {
            ASC->ApplyGameplayEffectSpecToTarget(*Spec.Data.Get(), TargetASC);
        }
        ASC->RemoveActiveEffects(Q);
```

```
    }
    else
    {
        // GE_BlazeStack：HasDuration 5s · AggregateByTarget · 上限 3 · 刷新持续时间 · 授予 Stat
        ApplyGameplayEffectToOwner(CurrentSpecHandle, CurrentActorInfo, CurrentActivationInfo,
                                   BlazeStackGE.GetDefaultObject(), GetAbilityLevel());
    }
}
```

网络分析

被动使用 `ServerOnly`：它只关心服务器上的命中结果，客户端不需要实例。层数通过 GE_BlazeStack 复制给拥有者（Mixed），Buff 栏可以显示"炽焰 ×2"；模拟端通过 Cue 看到刀身燃烧的特效，但不知道具体层数。如果对手也需要看到层数（例如显示在头顶），就要改成 Full 模式，或者用一个 COND_None 的属性记录层数。

14.6 Q · 冲锋斩：位移 + 免疫 + 预测

配置项	值
NetExecutionPolicy	LocalPredicted
AssetTags	`Ability.Yanren.Q`、`Ability.Type.Active`
ActivationOwnedTags	`State.Dashing`
BlockAbilitiesWithTag	`Ability.Type.Attack`、`Ability.Type.Active`（冲刺中不能普攻和放其他技能）
ActivationBlockedTags	`State.Debuff.Stun`、`State.Dead`
Cost / Cooldown	GE_Cost_Mana（50）/ GE_Cooldown（8 秒，`Cooldown.Yanren.Q`）

GA_Yanren_Q_Dash.cpp

```cpp
void UGA_Yanren_Q_Dash::ActivateAbility( ... )
{
    if (!CommitAbility(Handle, ActorInfo, ActivationInfo)) { EndAbility(Handle, ActorInfo, Act

    // ① 冲刺期间免疫控制：GE_Unstoppable 带 ImmunityComponent（免疫 AssetTag 含 Effect.Type.CC
    //     在预测窗口内施加 → 客户端也会预测得到 State.Buff.Unstoppable
    UnstoppableHandle = ApplyGameplayEffectToOwner(Handle, ActorInfo, ActivationInfo, Unstoppa

    // ② 位移：RootMotion Task，由 CMC 负责预测与校正
    const FVector Dir = GetAvatarActorFromActorInfo()->GetActorForwardVector();
    UAbilityTask_ApplyRootMotionConstantForce* Move = UAbilityTask_ApplyRootMotionConstantForc
        this, TEXT("Dash"), Dir, /*Strength*/ DashDistance / DashDuration, DashDuration,
        /*bIsAdditive*/ false, /*StrengthOverTime*/ nullptr,
        ERootMotionFinishVelocityMode::SetVelocity, FVector::ZeroVector, 0.f, /*bEnableGravity
    Move->OnFinish.AddDynamic(this, &ThisClass::OnDashFinished);
    Move->ReadyForActivation();

    // ③ 表现：拖尾 Cue（在预测窗口内 → 本地立即播放，服务器多播给其他人）
    K2_AddGameplayCue(MyTags::GameplayCue_Yanren_Q_Trail, MakeEffectContext(Handle, ActorInfo)

    // ④ 命中检测只在服务器：每帧扫掠胶囊体，对每个目标只结算一次
    if (HasAuthority(&ActivationInfo))
    {
        HitActors.Reset();
        GetWorld()->GetTimerManager().SetTimer(SweepTimer, this, &ThisClass::ServerSweep, 0.03
    }
}

void UGA_Yanren_Q_Dash::EndAbility( ... , bool bReplicateEndAbility, bool bWasCancelled)
{
    GetWorld()->GetTimerManager().ClearTimer(SweepTimer);
    // 移除免疫：GE 的移除不能预测，客户端的预测版本会在 Key 追平时自动清理
    BP_RemoveGameplayEffectFromOwnerWithHandle(UnstoppableHandle);
    Super::EndAbility(Handle, ActorInfo, ActivationInfo, bReplicateEndAbility, bWasCancelled);
}
```

> ⚠️ **注意**
>
> 扫掠计时器的回调**不在预测窗口内**，但它只在服务器上运行，服务器施加 GE 不需要预测 Key，所以没有问题。反过来，如果在这个计时器里给自己施加"需要预测"的 GE，客户端就不会预测（见 12.6）。

14.7 W • 晕眩飞刃：控制效果全链路

GE_Stun 的配置：

项	值	作用
Duration	HasDuration，1 秒（可按等级曲线）	
AssetTagsComponent	`Effect.Type.CC`	让免疫、驱散能识别它
TargetTagsComponent	`State.Debuff.Stun`	技能的 ActivationBlockedTags、CMC、动画蓝图都读这个 Tag
BlockAbilityTagsComponent	`Ability.Type.Active`、`Ability.Type.Attack`	眩晕期间阻止主动技能和普攻（被动不受影响）
GameplayCues	`GameplayCue.Status.Stun`	头顶转圈的星星（WhileActive 循环）



🌐 **被眩晕的玩家这边发生了什么**

1. 服务器施加 GE_Stun 后，经过 RTT/2，拥有者客户端收到 ActiveGE，本地加上 Stun Tag，UI 和动画进入眩晕状态。
2. 在这 RTT/2 内，玩家可能按了技能：客户端预测激活成功，服务器却因为 Stun 拒绝 → 回滚（这就是图 12-3 的场景）。
3. 打断由服务器执行 CancelAbilities，结果通过 ClientCancelAbility RPC 同步给拥有者；移动停止通过 CMC 的服务器校正同步。

图 14-3 控制效果的全链路，以及被控制方客户端上的时序

HeroMovementComponent.cpp —— 移动读取属性与 Tag

```cpp
float UHeroMovementComponent::GetMaxSpeed() const
{
    if (const UAbilitySystemComponent* ASC = UAbilitySystemGlobals::GetAbilitySystemComponentF
    {
        if (ASC->HasMatchingGameplayTag(MyTags::State_Debuff_Stun) || ASC->HasMatchingGameplay
        {
            return 0.f;
        }
        bool bFound = false;
        const float Speed = ASC->GetGameplayAttributeValue(UCombatSet::GetMoveSpeedAttribute()
        if (bFound) return Speed;
    }
    return Super::GetMaxSpeed();
}
```

14.8 R • 烈焰领域：地面选择 + 周期伤害

1. 技能激活后使用 `WaitTargetData`，TargetActor 选用 `GameplayAbilityTargetActor_GroundTrace`，ConfirmationType = **UserConfirmed**，地面显示范围指示器。
2. 玩家确认后，客户端把落点打包成 TargetData 发给服务器（13.4）；按右键取消则调用 `CancelAbility`，**此时还没有 Commit，所以不会进入冷却。**
3. 收到 TargetData 后 Commit（扣蓝、进冷却），服务器校验施法距离，然后生成复制的 `AFlameFieldActor`（持续 4 秒）。领域的视觉效果由这个 Actor 自己的 Niagara 组件负责，因为 Actor 本身就是复制的。
4. 领域 Actor 每 0.5 秒在服务器上检测范围内的敌人，施加 **GE_Burn**：HasDuration 1 秒、Period 0.5 秒、AggregateBySource、上限 1 层、刷新持续时间。站在领域里时持续时间不断被刷新；离开后最多再烧 1 秒。
5. GE_Burn 上挂 `GameplayCue.Status.Burn`（Looping），目标身上会出现火焰；伤害数字由每次周期执行时的跳字 Cue 显示。

✅ 为什么用"刷新的短 DOT"而不是"进入时施加、离开时移除"

"离开时移除"需要可靠地检测离开（目标死亡、传送、领域消失都要处理），而且 GE 的移除不能预测。用"持续时间很短、不断刷新"的 GE，在离开后自然过期，没有遗漏状态的风险，逻辑也更简单。

14.9 UI：技能栏冷却与 Buff 栏

技能按钮冷却显示

```
void USkillSlotWidget::NativeTick(const FGeometry& G, float Dt)
{
    Super::NativeTick(G, Dt);
    if (!ASC.IsValid()) return;

    const FGameplayEffectQuery Q = FGameplayEffectQuery::MakeQuery_MatchAnyOwningTags(Cooldowr
    float Remaining = 0.f, Duration = 0.f;
    for (const TPair<float, float>& P : ASC→GetActiveEffectsTimeRemainingAndDuration(Q))
    {
        if (P.Key > Remaining) { Remaining = P.Key; Duration = P.Value; }    // 取剩余时间最长的
    }
    // 平滑处理：避免 Key 追平时剩余时间小幅跳变（13.1）
    DisplayRemaining = FMath::FInterpTo(DisplayRemaining, Remaining, Dt, 20.f);
    SetCooldownPercent(Duration > 0.f ? DisplayRemaining / Duration : 0.f);
}
```

Buff 栏：绑定 ASC 的 `OnActiveGameplayEffectAddedDelegateToSelf`，并通过 `OnGameplayEffectRemoved_InfoDelegate(Handle)` 监听移除；用 `GE->FindComponent<UGameplayEffectUIData_TextOnly>()`（或项目自定义的 UIData 组件）读取图标和名称。没有 UIData 的 GE（例如冷却、消耗）不显示。

14.10 AbilitySet：一键授予整套技能

HeroAbilitySet.h / .cpp（参考 Lyra 的 ULyraAbilitySet）

```
USTRUCT(BlueprintType)
struct FHeroAbilityEntry
{
    GENERATED_BODY()
    UPROPERTY(EditDefaultsOnly) TSubclassOf<UGameplayAbility> Ability;
    UPROPERTY(EditDefaultsOnly) int32 Level = 1;
    UPROPERTY(EditDefaultsOnly, meta = (Categories = "InputTag")) FGameplayTag InputTag;
};

UCLASS()
class UHeroAbilitySet : public UPrimaryDataAsset
{
    GENERATED_BODY()
public:
    UPROPERTY(EditDefaultsOnly) TArray<FHeroAbilityEntry> Abilities;
    UPROPERTY(EditDefaultsOnly) TArray<TSubclassOf<UGameplayEffect>> Effects;

    void GiveToAbilitySystem(UAbilitySystemComponent* ASC, UObject* SourceObject, TArray<FGame
    {
        check(ASC && ASC->IsOwnerActorAuthoritative());          // 只能在服务器授予
        for (const FHeroAbilityEntry& E : Abilities)
        {
            FGameplayAbilitySpec Spec(E.Ability, E.Level);
            Spec.SourceObject = SourceObject;
            Spec.GetDynamicSpecSourceTags().AddTag(E.InputTag);  // 3.7 节的 InputTag 驱动
            OutHandles.Add(ASC->GiveAbility(Spec));
        }
        for (const TSubclassOf<UGameplayEffect>& GE : Effects)
        {
            ASC->ApplyGameplayEffectToSelf(GE.GetDefaultObject(), 1.f, ASC->MakeEffectContext(
        }
    }
};
```

14.11 每个技能的网络行为总表

技能	执行策略	客户端预测	只在服务器	模拟端看到
普攻	LocalPredicted	Montage、连段跳转、挥砍拖尾 Cue	扫掠检测、伤害、Landed 事件	Montage 复制、命中 Cue、血量变化
被动	ServerOnly	—	全部	刀身燃烧 Cue（随 GE）
Q	LocalPredicted	扣蓝、冷却、免疫 Tag、RootMotion、拖尾	路径扫掠、伤害	位移（CMC）、拖尾 Cue
W	LocalPredicted	扣蓝、冷却、出手动画	生成投射物、命中、眩晕	投射物 Actor、眩晕 Cue
R	LocalPredicted	地面指示器（LocalOnly 表现）、确认后的扣蓝冷却	距离校验、生成领域、周期伤害	领域 Actor、燃烧 Cue

14.12 联网测试清单

基础

☐ Dedicated Server + 2 个客户端

☐ 150ms 延迟 + 5% 丢包

☐ 中途加入的客户端能看到正在燃烧的目标（WhileActive）

预测

☐ 蓝量不足时按 Q：本地不扣蓝

☐ 冷却结束瞬间连续按键：没有闪烁、没有扣两次

☐ 被眩晕瞬间按键：回滚干净（蓝、冷却、动画、Cue）

打断与死亡

☐ 冲刺中被 W 命中：被免疫

☐ 普攻中被眩晕：技能被打断，EndAbility 被调用

☐ 死亡：清除 RemoveOnDeath 效果，重生后冷却保留

专用服务器

☐ 普攻的出手 AnimNotify 在服务器上能触发

☐ 服务器日志中没有 "No prediction key" 之类的警告

☐ `showdebug abilitysystem` 中两端的 Tag 一致

本章小结

- 英雄特有的只有技能流程；GE、计算类、Cue、AbilitySet 结构都是通用的，新增英雄的成本主要在配置。
- 伤害全部走"Spec + SetByCaller → ExecCalc → Meta 属性 → 结算"这一条管线。
- 技能之间通过 Tag 和 GameplayEvent 通信（普攻 → 被动、眩晕 → 打断），不直接互相引用。
- 每个技能都要明确：哪些在客户端预测、哪些只在服务器、模拟端通过什么看到。

CHAPTER 15

调试、性能与工程化

技能写出来只是开始。这一章讲如何快速定位问题、避免常见的性能陷阱，以及如何组织一个能长期维护的 GAS 工程。

15.1 调试工具箱

工具	用法	看什么
`showdebug abilitysystem`	控制台输入；`AbilitySystem.Debug.NextCategory` 切换类别；PageUp / PageDown 切换目标	属性、Owned Tags（含计数）、技能列表与激活状态、ActiveGE 与剩余时间
Gameplay Debugger	按 `'`（撇号键）打开，用小键盘数字切换到 GAS 类别	准星所指的 Actor 的 GAS 状态，服务器端数据也能看到
日志	`log LogAbilitySystem Verbose`、`log LogGameplayEffects Verbose`	激活失败原因、GE 施加与移除、预测键相关警告
Visual Logger	Tools → Debug → Visual Logger	按时间轴回放 GAS 事件（ASC 会记录部分 VLog）
控制台命令	输入 `AbilitySystem.` 后按 Tab 查看补全列表	新版本提供了授予技能、施加 GE、列出效果等调试命令（具体以版本为准）
Unreal Insights（Networking）	启动参数 `-trace=net`	每个 RPC 和属性的带宽占用，定位 GAS 的网络开销

✅ 排查"技能按了没反应"的固定顺序

1. `showdebug abilitysystem`：技能是否已授予？是否已处于激活状态（忘了 EndAbility）？
2. 看 Owned Tags：是否有 Cooldown Tag？是否有 ActivationBlockedTags 中的 Tag？是否被 BlockAbilitiesWithTag 阻挡？
3. 看属性：蓝量够不够 Cost？
4. 打开 `LogAbilitySystem Verbose`，查看 CanActivateAbility 失败的具体原因。
5. 只有客户端有问题？检查 ActorInfo 中的 PlayerController 是否为空（3.4）。

15.2 常见问题速查表

现象	原因	解决
技能只能放一次	某个出口没有调用 EndAbility	检查所有 Task 回调（Completed / Interrupted / Cancelled）
客户端预测技能不生效 / 全部等服务器	ActorInfo 中没有 PC；或者 NetExecutionPolicy 不是 LocalPredicted	PC 的 OnRep_PlayerState 中 RefreshAbilityActorInfo
属性在客户端偶尔算错	没有使用 REPNOTIFY_Always 或者没有调用 GAMEPLAYATTRIBUTE_REPNOTIFY	按 5.2 节修改
血条更新很慢，特效却很快	PlayerState 的 NetUpdateFrequency 太低	调高到 100 左右
Duration Buff 结束后数值没有恢复	在 PostGameplayEffectExecute 中直接改了属性；或者把 Buff 配成了带 Period 的 GE	持续加成只用 Modifier，不在回调中修改
DOT 不掉血	没有设置 Period	设置 Period
模拟端看不到动画	直接调用了 Montage_Play	使用 PlayMontageAndWait
专用服务器上技能卡住	AnimNotify 没有触发（动画不 Tick）	VisibilityBasedAnimTickOption 设置为 AlwaysTickPose…
后加入的玩家看不到持续特效	循环特效写在了 OnActive 中	改为 WhileActive
客户端调用 RemoveActiveGameplayEffect 后 Buff 又出现了	GE 的移除只能由服务器执行	在服务器上移除
Loose Tag 在其他客户端上没有	Loose Tag 默认不复制	使用复制版本的接口，或者改用 GE 授予
WaitGameplayEvent 在服务器上收不到客户端发出的事件	GameplayEvent 本身不同步	两端各自产生事件，或者使用 RPC（13.6）

冲刺被拉回	使用了 SetActorLocation / LaunchCharacter	使用 RootMotion Task
冷却 UI 跳变	预测版本与服务器版本的开始时间不同	平滑处理 + 输入缓冲（13.1）
同一个 AttributeSet 的属性在 GE 下拉中找不到 / 重复	AttributeSet 没有在 OwnerActor 上作为子对象创建，或者同类添加了两次	在 OwnerActor 构造函数中 CreateDefaultSubobject
Tag 改名后资产失效	没有配置重定向	添加 GameplayTagRedirects
伤害数字在客户端读到的 GEModData 为空	客户端收到的是属性复制，没有 GE 上下文	使用跳字 Cue（参数中带 Context）
AOE 命中很多目标时部分特效丢失	一帧内大量 Unreliable 多播	合并 Cue 或改用本地 Cue（10.5）

15.3 性能要点

GE 数量

每个 ActiveGE 都占用内存和复制开销。能用一个 GE 加 Stacking 实现的，不要创建多个独立 GE；能用 Tag 表示的简单状态，不需要一个空的 Infinite GE。

聚合器重算

不快照的 Attribute Based Modifier 会在源属性每次变化时重新计算。依赖链很长时（A → B → C），一次修改会引发多次重算，要控制派生属性的层数。

Tick 型 Task

`bTickingTask` 的 Task 每帧都会执行，大量单位同时使用时开销明显。能用事件或计时器解决的，不要用 Tick。

InstancedPerExecution

每次激活都创建一个 UObject，会增加 GC 压力。普攻这类高频技能必须使用 InstancedPerActor。

Cue 加载

GameplayCueManager 默认会加载配置目录下的所有 Cue。Cue 多的项目应限制

复制

AI 使用 Minimal；只有自己需要的属性设为 COND_OwnerOnly；Full 模式要先做带宽分析；

`GameplayCueNotifyPaths`，并按需异步加载（Lyra 的 `ULyraGameplayCueManager` 就是这样做的）。Actor 类 Cue 开启对象池复用。

PlayerState 始终相关这一点要纳入评估（13.10）。

15.4 参考架构：Lyra

Epic 的 Lyra 示例项目是目前最完整的 GAS 工程化参考。它的核心结构如下：

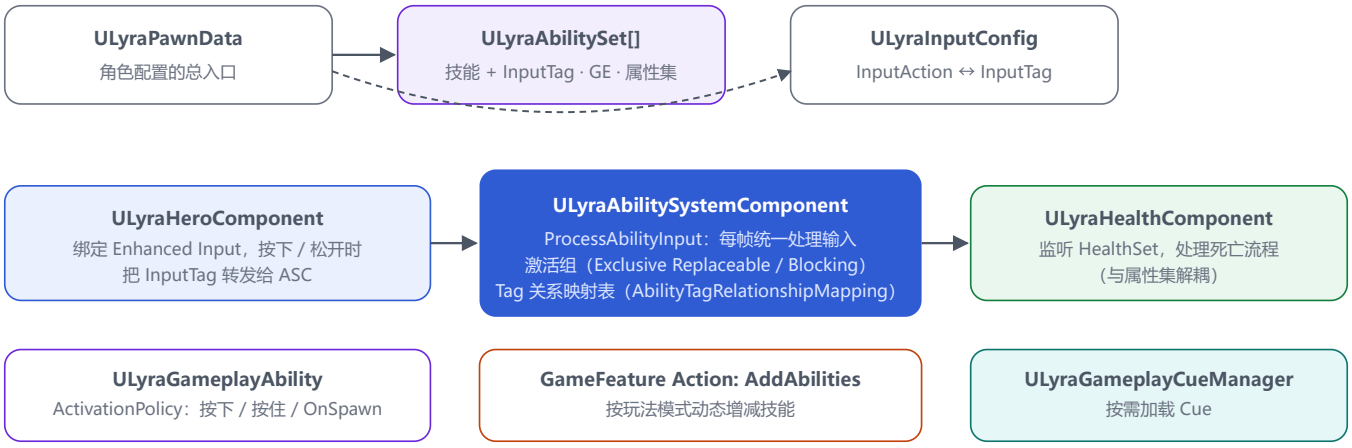


图 15-1 Lyra 的 GAS 结构（简化）。值得借鉴的三点：InputTag 输入、AbilitySet 资产、Tag 关系映射表

> ✅ **值得借鉴：Tag 关系映射表**
>
> Lyra 把"哪类技能阻挡、打断哪类技能"集中写在一个 DataAsset（`ULyraAbilityTagRelationshipMapping`）中，ASC 在激活时统一查询，而不是在每个技能上分别配置 Block / Cancel Tag。技能数量多了以后，这能避免规则分散、互相矛盾。

15.5 命名与目录规范

资产类型	前缀	示例
GameplayAbility	`GA_`	`GA_Yanren_Q_Dash`
GameplayEffect	`GE_`	`GE_Damage`、`GE_Cooldown`、`GE_Stun`
GameplayCue Notify	`GC_` / `GCN_`	`GC_Status_Burn`
AttributeSet	`AS_` 或 `*Set`	`UHealthSet`
ExecutionCalculation / MMC	`ExecCalc_` / `MMC_`	`ExecCalc_Damage`
AbilityTask	`AbilityTask_` / `AT_`	`AbilityTask_Charge`
AbilitySet	`DA_AbilitySet_`	`DA_AbilitySet_Yanren`

推荐的目录结构

```
Content/
└─ GAS/
   ├─ Common/              通用：GE_Damage / GE_Cooldown / GE_Stun / ExecCalc / MMC
   ├─ Cues/                所有 GameplayCue（GameplayCueNotifyPaths 指向这里）
   │  ├─ Status/           眩晕、燃烧、减速等状态表现
   │  └─ Heroes/Yanren/
   ├─ Heroes/
   │  └─ Yanren/           GA_Yanren_*、DA_AbilitySet_Yanren、英雄特有 GE
   └─ Curves/              CT_HeroStats（属性成长）、CT_AbilityScaling（技能数值）
Config/Tags/
   ├─ AbilityTags.ini      Ability.* / Cooldown.*
   ├─ StateTags.ini        State.* / Effect.*
   └─ CueTags.ini          GameplayCue.*
```

15.6 继续深入的方向

- **读源码**：按顺序阅读 `GameplayPrediction.h`（文件头注释）→ `AbilitySystemComponent_Abilities.cpp`（激活流程）→ `GameplayEffect.cpp` 中的 `FActiveGameplayEffectsContainer`（GE 的施加与聚合）。
- **读 Lyra**：重点看 `LyraAbilitySystemComponent`、`LyraGameplayAbility_RangedWeapon`、`LyraHealthSet`。

- **实践**：用本书第 14 章的结构再做一个风格完全不同的英雄（例如召唤、变身或者充能类技能），检验这套结构是否足够通用。

本章小结

- 排查问题先看 `showdebug abilitysystem`：技能、Tag、属性、GE 四项依次检查。
- 性能主要取决于 GE 数量、Tick Task、Cue 加载方式和复制模式。
- 工程化参考 Lyra：InputTag 输入、AbilitySet 资产、Tag 关系映射表、按需加载 Cue。

APPENDIX

附录　速查

术语表、API 速查、流程速查卡与参考资料。建议打印出来放在手边。

APPENDIX A

术语表

术语	说明
ASC	AbilitySystemComponent，GAS 的核心组件
OwnerActor / AvatarActor	ASC 所在的 Actor / ASC 在世界中作用的身体
Spec / Handle	运行时实例数据 / 指向实例的轻量 ID
BaseValue / CurrentValue	永久值 / 经过持续型 Modifier 聚合后的值
Aggregator	每个属性上的 Modifier 聚合器
Meta Attribute	不复制的临时中转属性，例如 IncomingDamage
MMC	ModifierMagnitudeCalculation，单个 Modifier 的数值计算类
ExecCalc	GameplayEffectExecutionCalculation，多输入多输出的计算类，仅服务器运行
SetByCaller	施加 GE 前由代码以 Tag 为键写入的数值
Snapshot	在 Spec 创建时捕获并固定属性值
Stacking	同一 GE 多次施加时的叠层规则
GEComponent	5.3+ GE 的可插拔行为组件
Loose Tag	不来自 GE 或技能、手动添加的 Tag，默认不复制
PredictionKey	客户端生成的预测 ID，用于关联预测副作用与服务器结果
Prediction Window	存在有效预测 Key 的代码作用域
CaughtUp / Rejected	预测 Key 被服务器确认追平 / 被拒绝
Replication Mode	Full / Mixed / Minimal，决定 GE 信息的复制范围
TargetData	可复制的目标信息容器
GameplayCue	由 Tag 驱动的表现层回调

APPENDIX B

API 速查

ASC：技能

`GiveAbility(Spec)` / `ClearAbility(Handle)`	授予 / 收回（服务器）
`TryActivateAbility(Handle)` / `TryActivateAbilitiesByTag(Tags)`	尝试激活
`CancelAbilities(WithTags, WithoutTags)` / `CancelAllAbilities()`	打断
`FindAbilitySpecFromHandle` / `GetActivatableAbilities()`	查询
`HandleGameplayEvent(Tag, Payload)`	发送事件（蓝图：SendGameplayEventToActor）

ASC：效果与属性

`MakeEffectContext()` / `MakeOutgoingSpec(Class, Level, Ctx)`	构造 Spec
`ApplyGameplayEffectSpecToSelf / ToTarget`	施加
`RemoveActiveGameplayEffect(Handle)` / `RemoveActiveEffects(Query)`	移除（服务器）
`GetActiveEffectsTimeRemainingAndDuration(Query)`	剩余时间
`GetAggregatedStackCount(Query)`	层数
`GetNumericAttribute(Attr)` / `SetNumericAttributeBase(Attr, V)`	读 / 写属性
`GetGameplayAttributeValueChangeDelegate(Attr)`	属性变化委托

ASC：Tag 与 Cue

`HasMatchingGameplayTag(Tag)` / `HasAnyMatchingGameplayTags` / `GetOwnedGameplayTags()`	查询
`AddLooseGameplayTag` / `RemoveLooseGameplayTag`	本地 Tag
`RegisterGameplayTagEvent(Tag, Type)`	Tag 变化委托
`ExecuteGameplayCue` / `AddGameplayCue` / `RemoveGameplayCue`	Cue（服务器多播）

GameplayAbility

`CanActivateAbility` / `ActivateAbility` / `ActivateAbilityFromEvent`（蓝图）	激活
`CommitAbility` / `CommitAbilityCost` / `CommitAbilityCooldown`	提交资源
`EndAbility` / `CancelAbility`	结束
`MakeOutgoingGameplayEffectSpec` / `ApplyGameplayEffectSpecToOwner / ToTarget`	施加 GE（自动带预测 Key）
`HasAuthority(&ActivationInfo)` / `IsPredictingClient()` / `IsLocallyControlled()`	判断执行端
`OnGiveAbility` / `OnAvatarSet` / `OnRemoveAbility`	生命周期钩子
`GetCooldownTags` / `ApplyCooldown` / `CheckCost`	可重写的资源逻辑

AttributeSet

`PreAttributeChange` / `PostAttributeChange`	Current 变化前后（任何来源）
`PreAttributeBaseChange` / `PostAttributeBaseChange`	Base 变化前后
`PreGameplayEffectExecute` / `PostGameplayEffectExecute`	Instant / Periodic 执行前后（仅服务器 + 预测端）
`GAMEPLAYATTRIBUTE_REPNOTIFY` / `DOREPLIFETIME_CONDITION_NOTIFY( ... , REPNOTIFY_Always)`	复制

APPENDIX C

流程速查卡

技能激活

TryActivate
→ CanActivate（Tag / Cost / CD / Block）
→ PreActivate（OwnedTags / Cancel / Block）
→ ActivateAbility
→ CommitAbility
→ Tasks…
→ EndAbility / CancelAbility

GE 施加

MakeOutgoingSpec（源快照）
→ SetByCaller
→ Apply → 免疫 / 条件 / 概率
→ Instant：Execute → 改 Base → Executed Cue
→ Duration：叠层 or 新建 ActiveGE
→ Modifier 入聚合器 → Tags →
OnActive/WhileActive

属性回调

PreGEExecute
→ PreAttributeBaseChange → Base
→ PreAttributeChange → Current
→ PostAttributeChange
→ PostGEExecute

预测

客户端：窗口生成 Key → RPC → 本地执行
服务器：用同一 Key 执行 → Succeed/Failed
→ 窗口结束写入 KeyMap
客户端：CaughtUp → 换成服务器版本
Rejected → 撤销全部副作用

聚合公式

((Base + ΣAddBase)
× (1+Σ(M−1)) ÷ (1+Σ(D−1))
× ΠCompound) + ΣAddFinal
Override 优先

复制模式

Full：所有人收到完整 GE
Mixed：拥有者完整，其他人 Tag + Cue
Minimal：所有人只有 Tag + Cue
玩家 Mixed · AI Minimal

APPENDIX D

参考资料

- Epic 官方文档：Gameplay Ability System for Unreal Engine
- Epic 社区教程：Gameplay Ability System - Best Practices for Setup（适用 5.4 / 5.5，含 5.5 / 5.6 的变更说明）
- Epic Python API：GameplayModOp（5.6）：聚合公式的官方定义
- tranek：GASDocumentation：社区最全面的 GAS 文档（主要基于 UE4.27 / 5.0，阅读时注意版本差异）
- Epic 示例：Lyra Sample Game
- 引擎源码（`Engine/Plugins/Runtime/GameplayAbilities/Source/GameplayAbilities/`）：
  - `Public/GameplayPrediction.h`：预测系统的设计说明（必读）
  - `Public/AbilitySystemComponent.h` 与 `Private/AbilitySystemComponent_Abilities.cpp`
  - `Public/GameplayEffect.h` 与 `Private/GameplayEffect.cpp`
  - `Public/GameplayEffectTypes.h`：EGameplayModOp、FGameplayEffectContext
  - `Public/AttributeSet.h`、`Public/Abilities/GameplayAbility.h`、`Public/GameplayCueManager.h`
  - `Public/GameplayEffectComponents/`：5.3+ 的所有 GE 组件

> 🕒 **最后提醒**
>
> 本书以 UE 5.4 – 5.6 为基准。GAS 每个版本都有调整，遇到书中 API 与项目引擎对不上的情况，请以项目中的引擎源码为准。头文件注释通常就是最准确的文档。